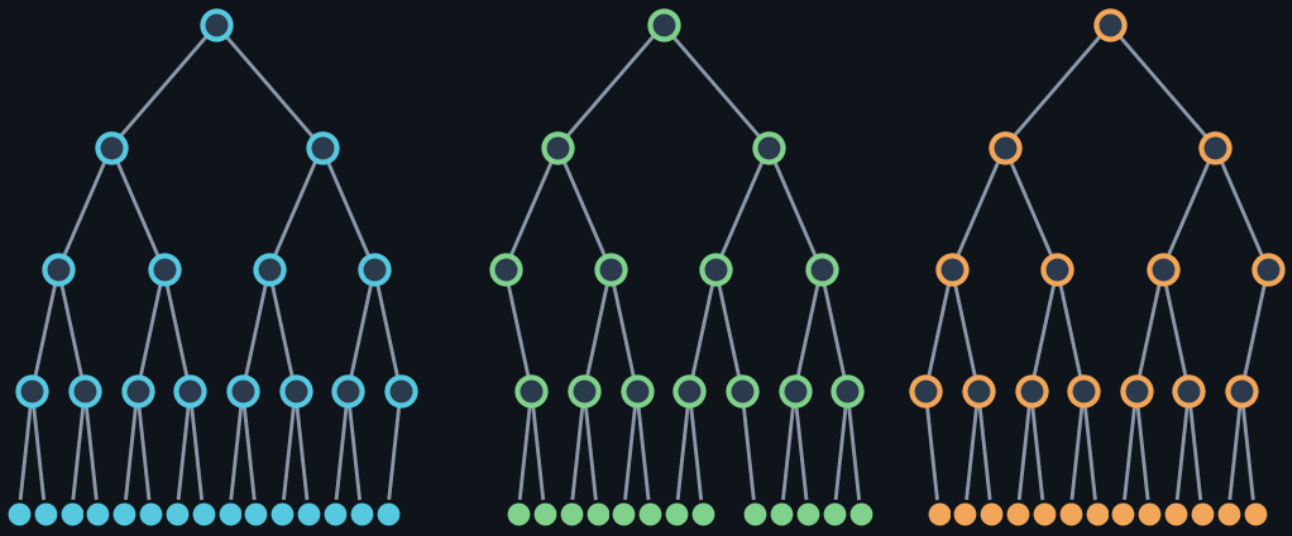
COMPLETE TUTORIAL • BASIC → PRO

Random Forest & Decision Tree Hyperparameter Tuning

From “what is a tree?” to GridSearchCV, pruning, speed tricks and professional workflows — explained with pictures, maths, examples and real experiments.



Built from your notes: every hyperparameter, every optimisation tip and your full GridSearchCV code are included — reorganised, explained and extended.
Dark theme for comfortable reading.

Contents

Part 1 — Foundations (Basic) 4

1.1 Parameters vs hyperparameters . . . 4
1.2 The decision tree from scratch . . . 4
1.3 Measuring a split: impurity, entropy and information gain . . . 5
1.4 Overfitting, underfitting and the bias–variance trade-off . . . 7

Part 2 — How a Random Forest Works (Core) 10

2.1 The wisdom of crowds . . . 10
2.2 Bagging: bootstrap + aggregate . . . 10
2.3 The second trick: random features at every split . . . 11
2.4 Putting it together: your first forest . . . 12

Part 3 — The 7 Random Forest Hyperparameters (Tuning) 13

3.0 The control panel at a glance . . . 13
3.1 n_estimators — the number of trees . . . 13
3.2 max_depth — how deep each tree may grow . . . 14
3.3 min_samples_split and min_samples_leaf — node and leaf constraints . . . 16
3.4 max_features — random features per split . . . 17
3.5 bootstrap — resampling the rows . . . 18
3.6 criterion — the split quality function . . . 19
3.7 How the knobs interact . . . 20

Part 4 — GridSearchCV: Tuning Automatically (Practice) 21

4.1 Why we need a search . . . 21
4.2 Cross-validation: how each combination is scored fairly . . . 21
4.3 Anatomy of GridSearchCV . . . 22
4.4 The grid-size maths . . . 22
4.5 Classification or regression? Choosing the scoring metric . . . 23
4.6 The complete GridSearchCV code for Random Forest . . . 24
4.7 Reading the results — a real run . . . 25

Part 5 — Optimising Decision Trees: Faster, Smaller, More Accurate 27

5.1 The easy version . . . 27
5.2 The technical version . . . 28
5.3 Your full Decision Tree GridSearchCV code . . . 31

Part 6 — Professional Workflow (Pro) 35

6.1 Beyond the full grid: smarter searches . . . 35
6.2 The coarse-to-fine workflow . . . 36
6.3 Pipelines, data leakage and nested cross-validation . . . 36
6.4 Out-of-bag score: free validation . . . 37
6.5 What did the forest learn? Feature importance . . . 37
6.6 Speed and memory playbook . . . 38
6.7 Common mistakes . . . 38

6.8 Master cheat-sheet . . . 39
6.9 Glossary . . . 40
6.10 Self-check quiz . . . 40
6.11 Summary and what to do next . . . 41

How to read this tutorial

The tutorial climbs in five levels. Each level only needs the ones before it, so if you are new, start at the top and do not skip. If you already know trees, jump to the level that matches you — every part starts with a one-paragraph recap.

Level	Part	What you will be able to do
1 · Basic	Part 1 — Foundations	Explain what a decision tree is, how it picks a split (Gini / entropy / MSE), and why it overfits.
2 · Core	Part 2 — Random Forest	Explain bagging, bootstrap samples, random feature choice, voting and why many trees beat one.
3 · Tuning	Part 3 — The 7 hyperparameters	Know what `n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features`, `bootstrap`, `criterion` really do, with measured charts.
4 · Practice	Part 4 — GridSearchCV	Run a grid search correctly: cross-validation, grid size maths, scoring metrics, reading results.
4 · Practice	Part 5 — Optimising decision trees	Make single trees faster, smaller and more accurate: pruning, binning, downcasting, class weights, histogram trees.
5 · Pro	Part 6 — Professional workflow	RandomizedSearchCV, halving search, pipelines, nested CV, OOB, importances, mistakes, cheat-sheet, quiz.

ABOUT THE EXPERIMENTS

Every chart marked “measured” in this tutorial was produced by actually running scikit-learn on a synthetic dataset (1,500 rows × 20 features) while this document was built, so the numbers are real, not invented. Your own data will give different numbers — the *shapes* of the curves are what matter.

ABOUT YOUR NOTES

A few of your notes list the same grid slightly differently (for example `n_estimators` appears as [100, 200, 300, 500] in one place and [100, 200, 500] in another). Both are kept and explained. Where something in the notes needed a small correction or clarification, it is marked with a “NOTE”.

Part 1 — Foundations (Basic)

1.1 Parameters vs hyperparameters

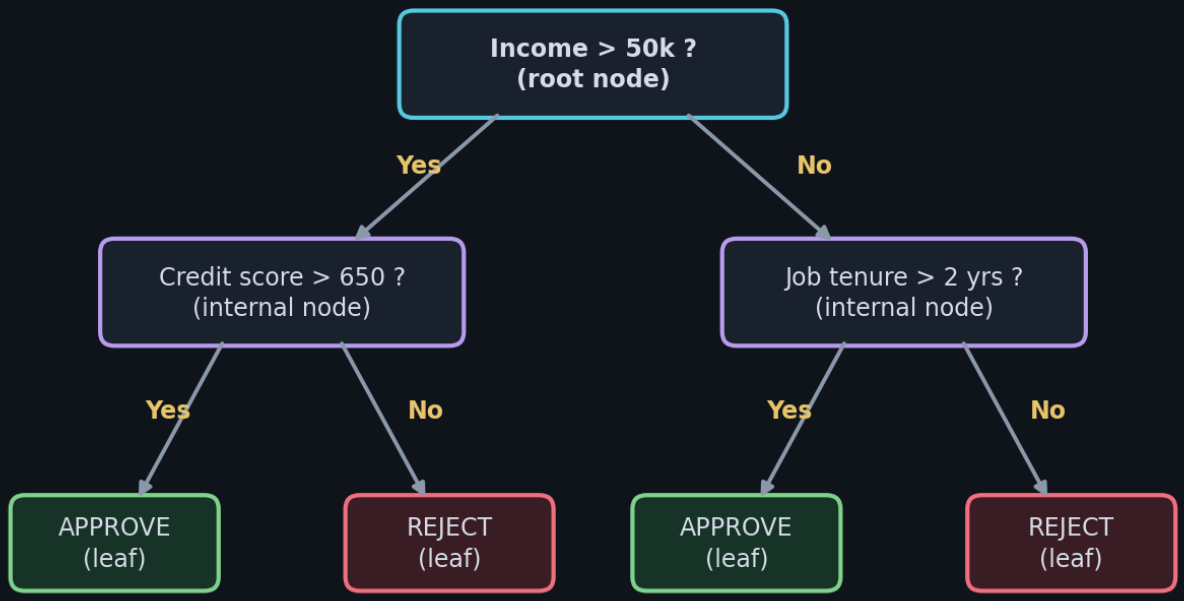
A machine-learning model has two kinds of numbers, and mixing them up is the number-one beginner confusion.

	Parameters	Hyperparameters
Who sets them?	The **algorithm learns** them from data.	**You choose** them before training.
Examples (tree)	Which feature to split on, the threshold (age > 30), the value stored in each leaf.	`max_depth`, `n_estimators`, `min_samples_leaf` …
Analogy	The recipe steps a chef discovers while cooking.	The oven temperature and cooking time you set before starting.
How found?	Training (`.fit()`).	Trial and error — this is exactly what **GridSearchCV** automates.

ANALOGY
Think of baking bread. The **hyperparameters** are oven temperature, baking time and how much yeast you add. The **parameters** are what happens inside the dough. You cannot reach inside and set the dough directly — you can only choose the settings and taste the result. Tuning means trying several settings and keeping the one that gives the best bread (the best score on data the model has never seen).

1.2 The decision tree from scratch

A decision tree is the “20 Questions” game played by a computer. It asks a chain of yes/no questions about your data, and the answers lead you to a final prediction. Here is a tiny tree that decides whether a bank approves a loan:



Every person walks from the root down to ONE leaf — the leaf is the prediction.

Figure 1 — A decision tree. Boxes with questions are nodes; the coloured end boxes are leaves.

Term	Meaning	Related hyperparameter
Root node	The first question; contains all training data.	—
Internal node	A question that splits the data into two groups.	`min_samples_split`, `max_features`, `criterion`
Branch	One answer (Yes / No) leaving a node.	—
Leaf node	An end box that outputs the prediction (a class or a number).	`min_samples_leaf`
Depth	The number of questions on the longest root-to-leaf path. Figure 1 has depth 2.	`max_depth`

How does the tree choose its questions?

The tree is **greedy**. At every node it tries many candidate questions — every feature, and for numeric features many threshold values — and picks the one that makes the two resulting groups as *pure* as possible (mostly “Yes”, or mostly “No”). Then it repeats the process on each group, until it hits a stopping rule. To score “how pure”, it needs a number. That number is called **impurity**.

1.3 Measuring a split: impurity, entropy and information gain

Suppose a node holds samples from classes 1…C, and p_i is the fraction belonging to class i. Two classic impurity formulas are used by scikit-learn (these are the `criterion` values in your notes):

$$G = 1 - \sum_{i=1}^{C} p_i^2$$

$$H = -\sum_{i=1}^{C} p_i \log_2 p_i$$

Top: Gini impurity (criterion='gini'). Bottom: Entropy (criterion='entropy').

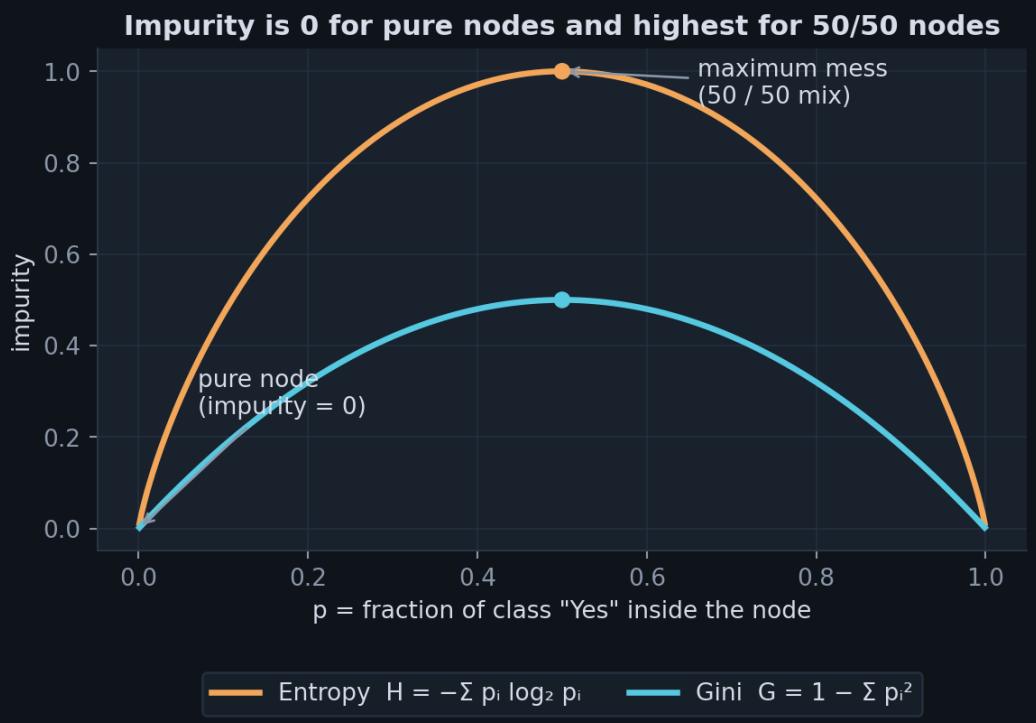


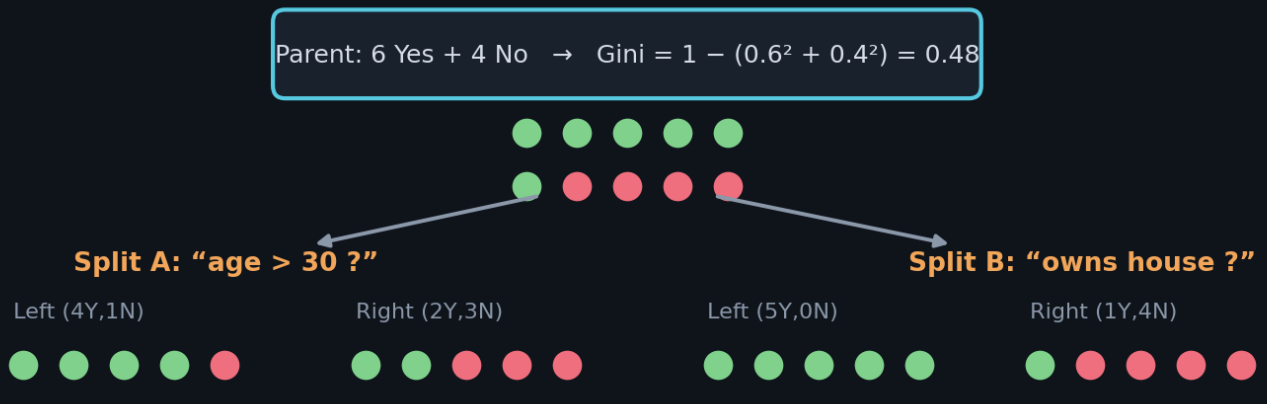
Figure 2 — Both measures are 0 for a pure node and largest for a 50/50 mix. They behave almost identically, which is why accuracy rarely differs between them.

A split creates a left child (N_L samples) and right child (N_R samples) from a parent of N samples. The quality of the split is the **gain** — how much impurity drops, averaging the two children by their size:

$$\text{Gain} = I(\text{parent}) - \left[\frac{N_L}{N} I(\text{left}) + \frac{N_R}{N} I(\text{right})\right]$$

Use I = Gini or I = Entropy. With entropy, this gain is called **information gain**.

Worked example. A parent node has 10 people: 6 “Yes” and 4 “No”. Two candidate questions are compared:



Gini 0.32 | Gini 0.48
Weighted = 0.5·0.32 + 0.5·0.48 = 0.40
Gain = 0.48 − 0.40 = 0.08

Gini 0.00 | Gini 0.32
Weighted = 0.5·0.00 + 0.5·0.32 = 0.16
Gain = 0.48 − 0.16 = 0.32 ✔ WINNER

Figure 3 — Split B produces one pure child, so its gain (0.32) beats split A (0.08). The tree picks B.

$$G_{parent} = 1 - (0.6^2 + 0.4^2) = 0.48$$

$$\text{Entropy}_{parent} = -(0.6\log_2 0.6 + 0.4\log_2 0.4) \approx 0.971$$

With entropy: split A gives gain ≈ 0.125 and split B gives gain ≈ 0.610 — same winner.

Regression trees: variance instead of purity

When the target is a number (house price), "pure" means "all values in the node are close together". Regression trees therefore use error-based criteria — the other two `criterion` values in your notes:

$$\text{MSE} = \frac{1}{n}\sum_{j=1}^{n}(y_j - \bar{y})^2 \qquad \text{MAE} = \frac{1}{n}\sum_{j=1}^{n}|y_j - \text{median}(y)|$$

`squared_error` minimises MSE around the mean; `absolute_error` minimises MAE around the median. A leaf predicts the mean (or median) of its samples.

1.4 Overfitting, underfitting and the bias–variance trade-off

If you let a tree keep splitting until every leaf is pure, it can memorise the training set perfectly — including random noise. That is **overfitting**. If you stop too early, the tree is too simple to capture the pattern — **underfitting**.

ANALOGY
Two students prepare for an exam. One *memorises* last year's answers word for word (overfit: perfect on old questions, lost on new ones). The other learned only the chapter titles (underfit: weak everywhere). The goal is the student who *understood the ideas* (generalises).

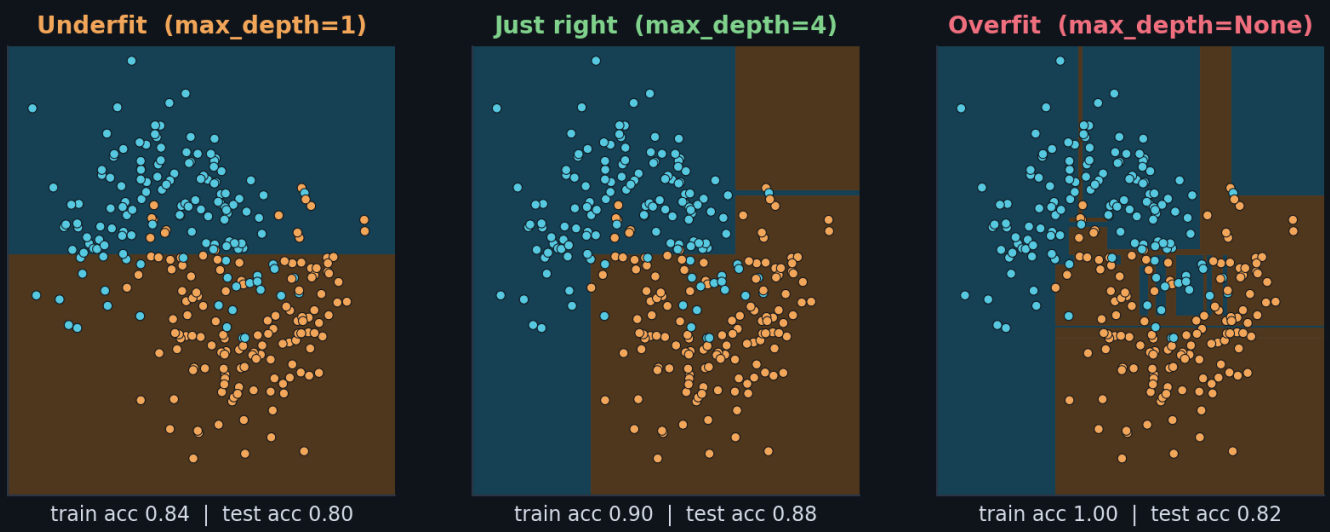


Figure 4 — Same data, three tree depths. Depth 1 is too crude, unlimited depth draws jagged islands around noise, depth 4 is a good compromise.

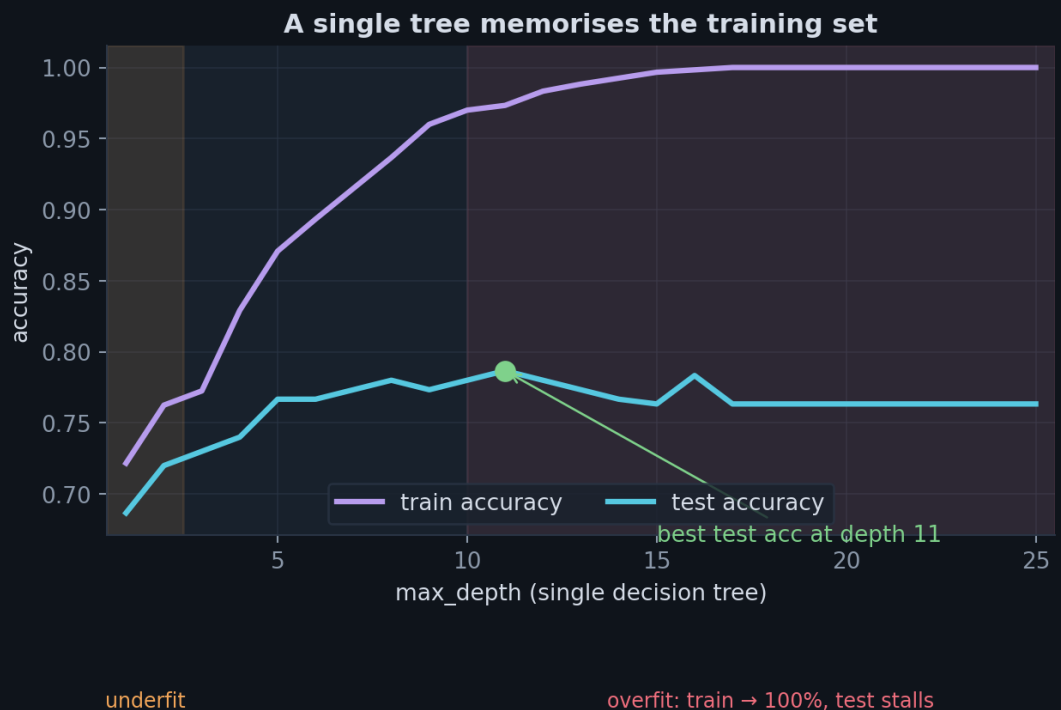


Figure 5 (measured) — Train accuracy reaches 100% with an unlimited tree, while test accuracy is only 76.3%. The gap is overfitting.

Statisticians split the expected test error into three parts:

$$\mathrm{E}\left[(y-\hat{y})^2\right] = \mathrm{Bias}^2 + \mathrm{Variance} + \sigma^2_{noise}$$

Bias = error from being too simple. Variance = error from being too sensitive to the particular training sample. Noise = unavoidable randomness.

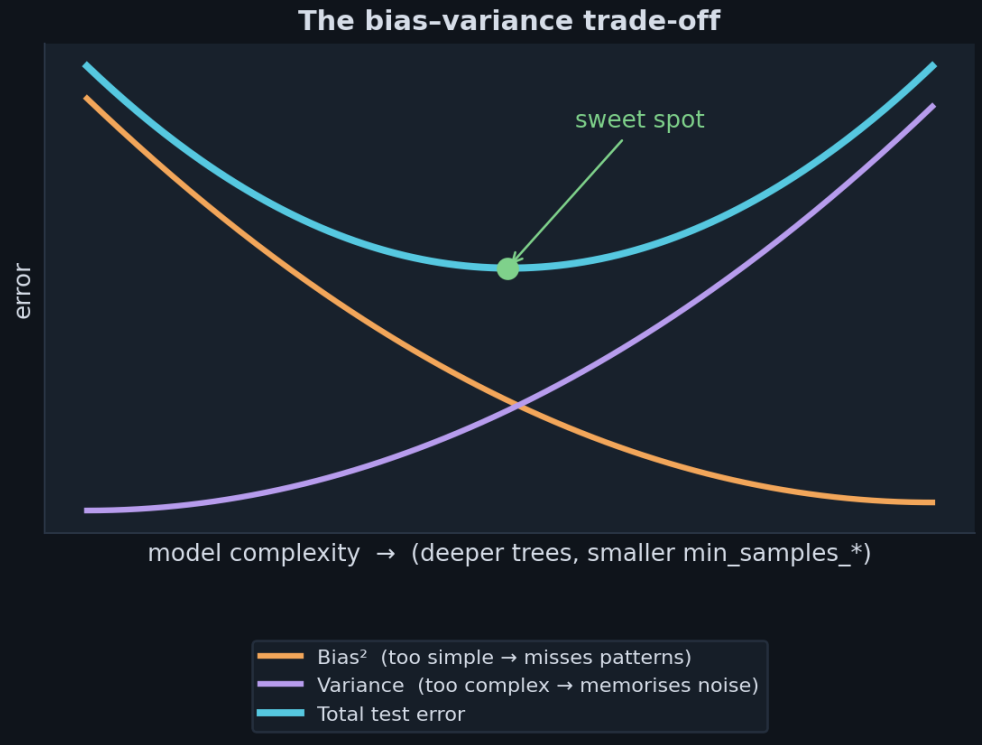


Figure 6 — Making a model more complex lowers bias but raises variance. Tuning hyperparameters is the search for the bottom of the blue curve.

KEY IDEA

A single deep decision tree has **low bias but very high variance**: change a few training rows and the whole tree can look different. Random Forests were invented to attack exactly this high variance — which is the story of Part 2.

Part 2 — How a Random Forest Works (Core)

2.1 The wisdom of crowds

Imagine guessing the number of jelly beans in a jar. One person's guess can be far off. But the *average* of 500 independent guesses is usually remarkably close, because individual errors point in different directions and cancel out. A Random Forest applies this idea to decision trees: instead of trusting one unstable tree, build many different trees and let them vote.

> **KEY IDEA**
> A Random Forest = **many decision trees** trained on **different random versions** of the data and features, whose predictions are **combined**. "Different" is the key word: if all trees were identical, voting would add nothing.

2.2 Bagging: bootstrap + aggregate

Random Forests make trees different in two ways. The first is **bagging** (**b**ootstrap **agg**regat**ing**):

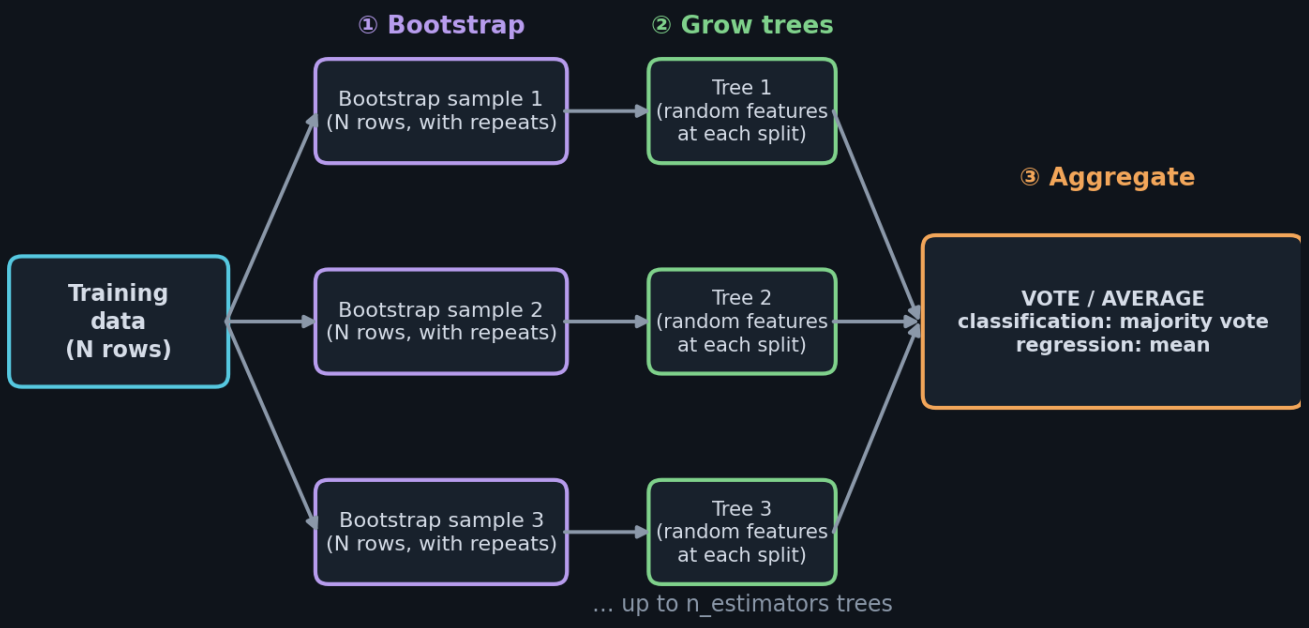


Figure 7 — Bagging: each tree sees its own bootstrap sample; the forest aggregates all tree outputs.

- **Bootstrap:** from N training rows, draw N rows *with replacement*. Some rows appear several times, some not at all.
- **Train** one tree per bootstrap sample (this is the `bootstrap=True` setting in your notes).
- **Aggregate:** classification → majority vote; regression → average.

$$\hat{y}_{class}(x) = \text{mode}\{h_1(x), \ldots, h_B(x)\}$$

$$\hat{y}_{reg}(x) = \frac{1}{B}\sum_{b=1}^{B} h_b(x)$$

$h_b(x)$ is the prediction of tree b, and B = `n_estimators` is the number of trees.

Why do about 63% of rows end up in each sample?

In one draw, a particular row is *not* chosen with probability (1 − 1/N). After N independent draws it is never chosen with probability $(1 - 1/N)^N$, which converges to 1/e as N grows:

$$P(\text{row never picked}) = \left(1 - \frac{1}{N}\right)^N \longrightarrow e^{-1} \approx 0.368 \quad (N \to \infty)$$

$$P(\text{row in bag}) \approx 1 - 0.368 = 0.632$$



Figure 8 — Left: the in-bag fraction converges to 63.2%. Right: a concrete bootstrap sample of 10 rows with duplicates; rows 4, 6, 8 are left out.

PRO TIP

The ≈ 36.8% of rows a tree never sees are its **out-of-bag (OOB)** rows. They act as a free validation set: predict each row using only the trees that did not train on it, and average the accuracy. In scikit-learn: `RandomForestClassifier(oob_score=True)` then read `.oob_score_`. It needs `bootstrap=True`.

2.3 The second trick: random features at every split

Bagging alone still produces similar trees, because one very strong feature would be chosen at the top of almost every tree. So Random Forest adds a second source of randomness: at **every node**, the tree may only look at a random subset of the features (this is `max_features`). A strong feature is then simply *unavailable* at many nodes, forcing trees to find other useful splits and making them diverse.

How much does diversity help? Suppose each tree has prediction variance σ^2 and any two trees have correlation ρ. The variance of the average of B trees is:

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B} h_b\right) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2$$

First term: the floor that more trees cannot remove. Second term: shrinks as B grows.

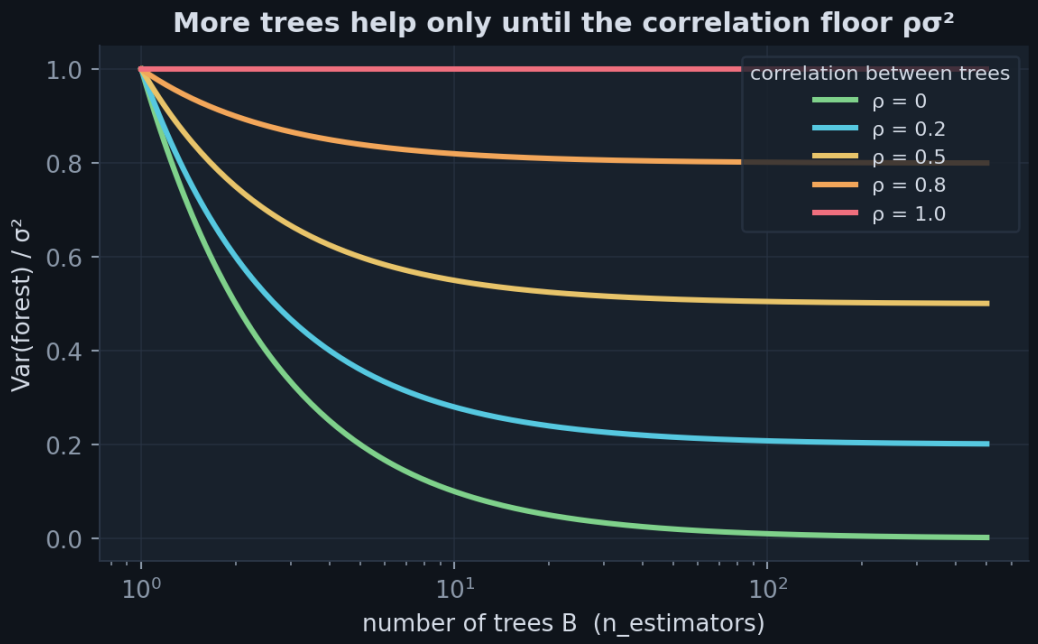


Figure 9 — With uncorrelated trees (ρ = 0) variance falls as 1/B. With highly correlated trees (ρ → 1) adding trees barely helps. Lowering ρ (random features!) is as important as adding trees.

THE FOREST IN ONE FORMULA

This single formula explains four of your hyperparameters:

- `n_estimators` ↑ shrinks the second term (but not the floor ρσ²).
- `max_features` ↓ lowers ρ (more diverse trees) but may raise each tree's own error.
- `bootstrap=True` lowers ρ via different data samples.
- `max_depth` / `min_samples_*` change how big each tree's own variance σ² and bias are.

2.4 Putting it together: your first forest

Python — a default Random Forest with an out-of-bag score

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

X, y = make_classification(n_samples=1500, n_features=20, n_informative=6,
                           n_redundant=4, flip_y=0.05, class_sep=0.8, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

rf = RandomForestClassifier(n_estimators=100, oob_score=True, random_state=42)
rf.fit(X_train, y_train)

print("OOB accuracy :", round(rf.oob_score_, 4))   # free validation estimate
print("Test accuracy:", round(rf.score(X_test, y_test), 4))
```

On this dataset this prints an OOB accuracy of about **0.867** (measured with 300 trees) and a test accuracy near **0.837**. Compare this with the single unlimited tree of Figure 5, which managed only 0.763 — the forest already beats it before any tuning.

Part 3 — The 7 Random Forest Hyperparameters (Tuning)

You now know what a forest is. This part opens the control panel. Each hyperparameter below gets the same treatment: **what it is** (your notes), **how it works**, **what it does to the model**, a **measured chart** from a real experiment, and the **grid values** to search.

3.0 The control panel at a glance

Hyperparameter	Default (classifier)	Grid in your notes	One-line job
`n_estimators`	100	[100, 200, 300, 500] or [100, 200, 500]	How many trees vote.
`max_depth`	None	[None, 10, 20, 30]	How deep each tree may grow.
`min_samples_split`	2	[2, 5, 10]	Smallest node that is allowed to split.
`min_samples_leaf`	1	[1, 2, 4]	Smallest allowed final leaf.
`max_features`	'sqrt'	['sqrt', 'log2', None]	How many features each split may consider.
`bootstrap`	True	[True, False]	Whether each tree gets a resampled dataset.
`criterion`	'gini'	['gini', 'entropy'] (regression: ['squared_error', 'absolute_error'])	How a split is scored.

REMEMBER IT AS 4 GROUPS

Group them by job and the list becomes easy to remember:

Crowd size → `n_estimators`. **Tree size limits** → `max_depth`, `min_samples_split`, `min_samples_leaf`. **Randomness / diversity** → `max_features`, `bootstrap`. **Scoring rule** → `criterion`.

NOTE

Defaults shown are for `RandomForestClassifier` in scikit-learn. For `RandomForestRegressor`, the defaults of `criterion` and `max_features` differ (`'squared_error'` and `1.0` = all features).

3.1 n_estimators — the number of trees

FROM YOUR NOTES

Number of Trees (n_estimators): The total number of individual decision trees built in the forest. More trees improve overall model performance and stability, but they linearly increase the training time.

Grid search values: [100, 200, 300, 500] — diminishing returns on performance usually kick in after 300 to 500 trees.

Mechanism

This parameter dictates the total volume of independent decision trees generated within the ensemble before the algorithm aggregates their individual predictions via majority vote or mathematical averaging.

Impact

Increasing this number stabilises the final model and smooths out predictions, effectively neutralising the high variance that plagues standalone decision trees. Because each tree is trained independently, adding more trees does not typically cause the model to overfit. However, it acts as a direct multiplier on computational cost, training time and memory usage.

$$T_{train} \approx B \cdot T_{one\ tree} \qquad \text{Memory} \approx B \cdot \text{size(one tree)}$$

B = `n_estimators`. Doubling the trees roughly doubles training time, prediction time and memory.

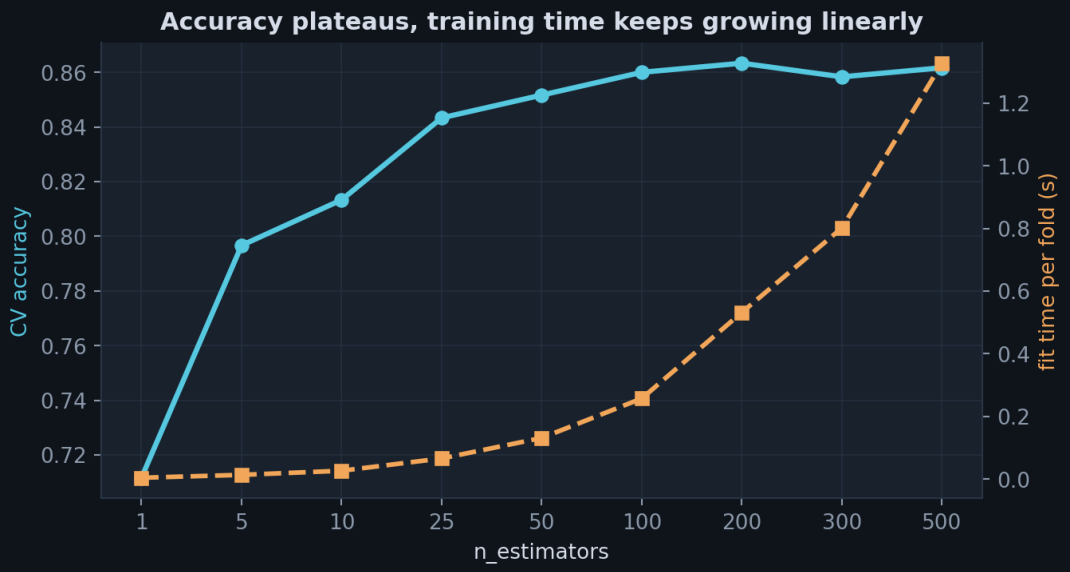


Figure 10 (measured) — CV accuracy rises from 71.2% (1 tree) to 86.0% (100 trees) and then plateaus around 86.2% at 500 trees, while the time per fit grows from 0.26s to 1.33s.

Grid search setup

[100, 200, 500] ensures you test the default scikit-learn baseline against sufficiently large forests where predictive performance gains typically begin to plateau. The longer list **[100, 200, 300, 500]** adds one more point on the plateau if you can afford the time.

PRO TIP

Tune everything else with a moderate forest (100–200 trees), then raise `n_estimators` only for the final model. It is the one hyperparameter that is "more is safer", so it does not need an expensive search — it only needs a budget decision. Also use `n_jobs=-1` so trees train in parallel.

3.2 max_depth — how deep each tree may grow

FROM YOUR NOTES

Maximum Depth (max_depth): How deep each individual tree is allowed to grow. Deep trees can capture highly complex patterns but are extremely prone to overfitting (memorising the training data instead of learning general rules).

Grid search values: [None, 10, 20, 30] — None lets trees grow indefinitely until all leaves are pure; 10 to 30 forces the model to generalise better on unseen data.

Mechanism

This establishes a strict, hard cutoff for how many consecutive levels deep a tree is permitted to grow from the initial root node down to the terminal leaves.

Impact

This is the single most critical lever for combating model memorisation. Leaving depth unbounded allows the model to perfectly map the training data, capturing microscopic noise and causing severe variance (overfitting) when applied to unseen test sets. Conversely, forcing the trees to remain overly shallow drastically increases bias, meaning the model fails to capture underlying complexity and underfits the problem.

Why does depth matter so much? A binary tree of depth *d* can have up to 2^d leaves — each leaf is a "rule". The capacity explodes quickly:

max_depth	Max. leaves (rules)	Meaning
3	8	Very simple, easily underfits
10	1,024	Moderate — good starting cap
20	1,048,576	More rules than most datasets have rows
30	≈ 1.07 billion	Effectively unlimited — same as `None` on small data
None	until leaves are pure	Memorises the training set

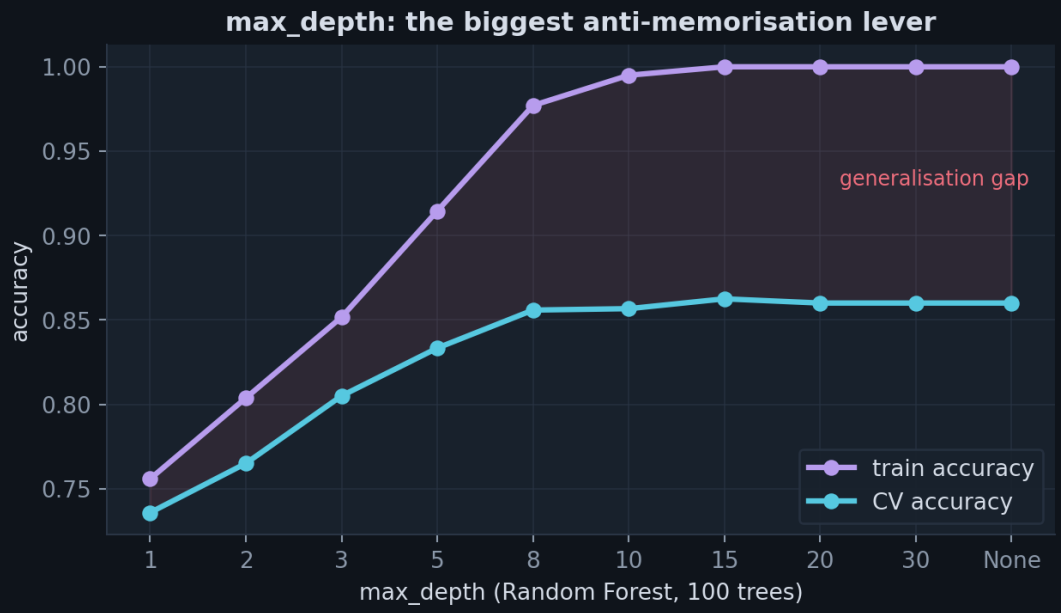


Figure 11 (measured) — Train accuracy hits ≈100% from depth 15, while CV accuracy flattens at ≈86.2%. Shallow forests (depth 1–3) clearly underfit.

WATCH OUT

Notice something subtle: in a **forest** the extra depth beyond ≈ 10–15 does not *hurt* CV accuracy here (85.7% at depth 10, 86.0% unlimited), even though training accuracy is 100%. Averaging many deep trees cancels much of their individual overfitting. A *single* tree (Figure 5) is far more sensitive. So `max_depth` matters most for **single trees**, for **speed and memory**, and on noisy data — always check with CV rather than assuming.

Grid search setup

[10, 20, 30, None] lets the algorithm evaluate generalised models against an unconstrained (`None`) baseline. If `None` wins, you do not need a depth cap for accuracy; if 10 wins, your data is noisy and you should also test smaller caps such as 5 or 8.

3.3 min_samples_split and min_samples_leaf — node and leaf constraints

FROM YOUR NOTES

min_samples_split (Minimum Samples for a Split): The minimum number of data points required in an internal node before the algorithm is allowed to split it further into branches.

Grid: [2, 5, 10] — 2 is the default, but increasing it to 5 or 10 stops the tree from creating highly specific, overfitted splits based on small data subsets.

min_samples_leaf (Minimum Samples per Leaf): The absolute minimum number of data points allowed at the very end of a branch (a leaf node).

Grid: [1, 2, 4] — 1 is the default; increasing to 2 or 4 smooths out the model's predictions, reduces noise, and prevents the model from isolating single outlier data points.

Mechanism

`min_samples_split` dictates the minimum dataset size required to divide an internal node, while `min_samples_leaf` sets the absolute floor for the number of data points permitted in a final terminal node.

Impact

Both parameters function as powerful structural regularisers that force trees to make broader generalisations. Elevating these values prevents the algorithm from isolating single outlier records into highly specific branches, which proactively reduces overfitting. However, excessively aggressive thresholds can strip the model of its ability to learn nuanced boundaries, resulting in underfitting.

ANALOGY

min_samples_split is a manager who refuses to hold a meeting about a group smaller than *k* people ("too few to bother dividing"). **min_samples_leaf** is a rule that *every final team must have at least k members*, no matter how the meeting is split. The second rule is stricter: it can veto a split even when the node is big enough.

$$\text{node can split} \Leftrightarrow n \geq \text{min_samples_split}$$

$$\text{and a split exists with both children} \geq \text{min_samples_leaf}$$

So a node with n samples needs $n \geq 2 \times$ `min_samples_leaf` before it can ever be split.

Example. A node has 6 samples, `min_samples_split=5` and `min_samples_leaf=4`. The node is large enough to be considered ($6 \geq 5$), but the best it could do is a 3 + 3 split — both children are smaller than 4 — so the split is rejected and the node becomes a leaf.

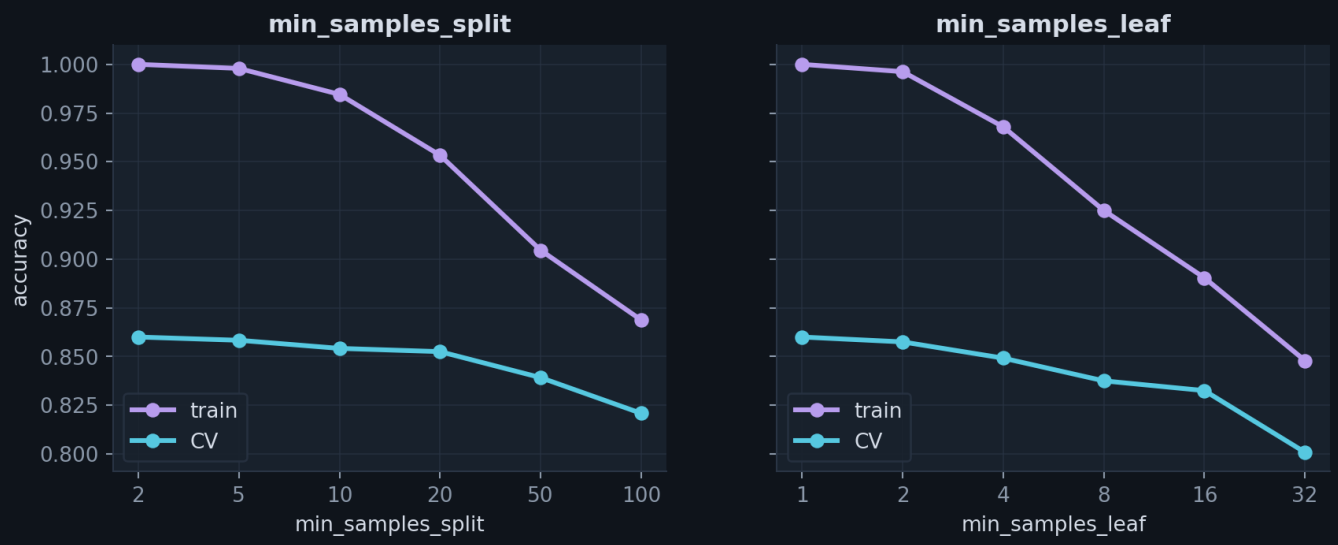


Figure 12 (measured) — Raising either value moves the forest from memorising (train = 100%) towards smoothing. Mild values (2–10) barely cost accuracy; aggressive values (50+, 16+) start to underfit.

Grid search setup

[2, 5, 10] for splits and **[1, 2, 4]** for leaves will systematically test the threshold between capturing valid patterns and ignoring noise. Both also accept a *fraction* (for example `min_samples_leaf=0.01` = 1% of the training rows), which scales automatically with dataset size.

3.4 max_features — random features per split

FROM YOUR NOTES

Maximum Features per Split (max_features): The number of variables the tree is allowed to randomly consider when evaluating the best way to split a node. Randomly restricting these variables is what makes each tree in the forest mathematically unique from the others.

Grid: ['sqrt', 'log2', None] — 'sqrt' is standard for classification tasks; None forces the tree to look at every single feature for every split, which removes the “random” element of the feature selection.

Mechanism

This parameter randomly restricts the pool of variables the algorithm is permitted to evaluate when determining the optimal mathematical split at any given internal node.

Impact

This forced constraint guarantees that individual trees remain structurally diverse and uncorrelated, which is the core mathematical advantage of a Random Forest. Smaller feature subsets reduce variance and prevent dominant features from overshadowing minor ones, though setting it too low can artificially inflate bias.

What do the options mean? With p total features:

$$m_{sqrt} = \lfloor \sqrt{p} \rfloor \qquad m_{log2} = \lfloor \log_2 p \rfloor \qquad m_{None} = p$$

m = how many random features each split may look at.

Total features p	'sqrt'	'log2'	None (all)
20	4	4	20
100	10	6	100
400	20	8	400

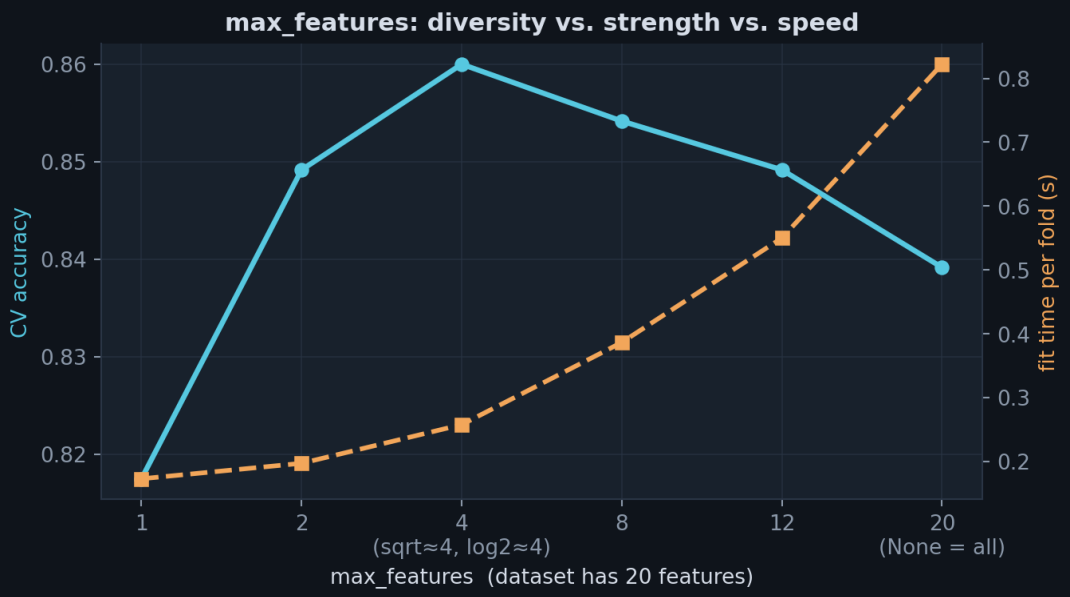


Figure 13 (measured) — With 20 features, ≈ 4 features per split (the `sqrt` / `log2` choice) gives the best CV accuracy (86.0%). Using all 20 is both *slower* (0.82s vs 0.26s per fit) and slightly less accurate (83.9%).

WATCH OUT

Setting `max_features=None` turns a Random Forest into plain **bagged trees**: the trees become more correlated (higher ρ in the variance formula) and each split costs more because every feature is examined. Setting it *too low* (like 1) forces trees to split on possibly useless features, so each tree gets weak — visible at the far left of Figure 13.

Grid search setup

['sqrt', 'log2', None] covers the standard algorithmic recommendations and tests them against `None`, which completely removes the randomness by forcing the tree to evaluate every single available feature. You can also pass an integer or a fraction (such as 0.5) to explore values in between.

3.5 bootstrap — resampling the rows

FROM YOUR NOTES

Bootstrapping (bootstrap): Whether each tree is trained on a random sample of the data with replacement (meaning some data rows are picked multiple times while others are left out entirely). If set to False, every tree trains on the exact same full dataset.

Grid: [True, False] — True is the default and typically yields the best results, but False is occasionally worth testing on smaller datasets.

You met the bootstrap in Figure 8. With `bootstrap=True`, each tree sees ≈ 63.2% distinct rows (some duplicated), which makes trees different from each other and gives you the free **OOB score**. With `bootstrap=False`, every tree sees all rows; the only remaining source of diversity is the random feature choice (`max_features`).

PRO TIP

In our measured test, `bootstrap=False` scored 87.3% vs 86.0% for `True` on 1,500 rows, and 78.7% vs 77.2% on a 200-row subset. The gap is about one point — small enough to be partly noise — but it shows exactly why your notes say “False is occasionally worth testing on smaller datasets”. Remember the trade-off: with `False` you lose the OOB score and some diversity.

3.6 criterion — the split quality function

FROM YOUR NOTES

Split Quality Function (criterion): The mathematical metric the model uses to measure how effective a split is at separating the target data.

Grid (classification): ['gini', 'entropy']

Grid (regression): ['squared_error', 'absolute_error']

These are the impurity formulas from Part 1 (Gini, entropy, MSE, MAE). Practical differences:

Criterion	Task	Notes
`gini`	Classification	Default. No logarithm, so slightly faster. Tends to isolate the largest class.
`entropy`	Classification	Uses $\log_2$; slightly more balanced splits. Usually the same accuracy as Gini.
`squared_error`	Regression	Default. Minimises MSE; leaf = mean. Sensitive to outliers.
`absolute_error`	Regression	Minimises MAE; leaf = median. Robust to outliers but **much slower** (needs medians).
Extra: `log_loss`, `friedman_mse`, `poisson`	Both	`log_loss` equals entropy for classification. `friedman_mse` is a faster MSE variant. `poisson` is for count targets.

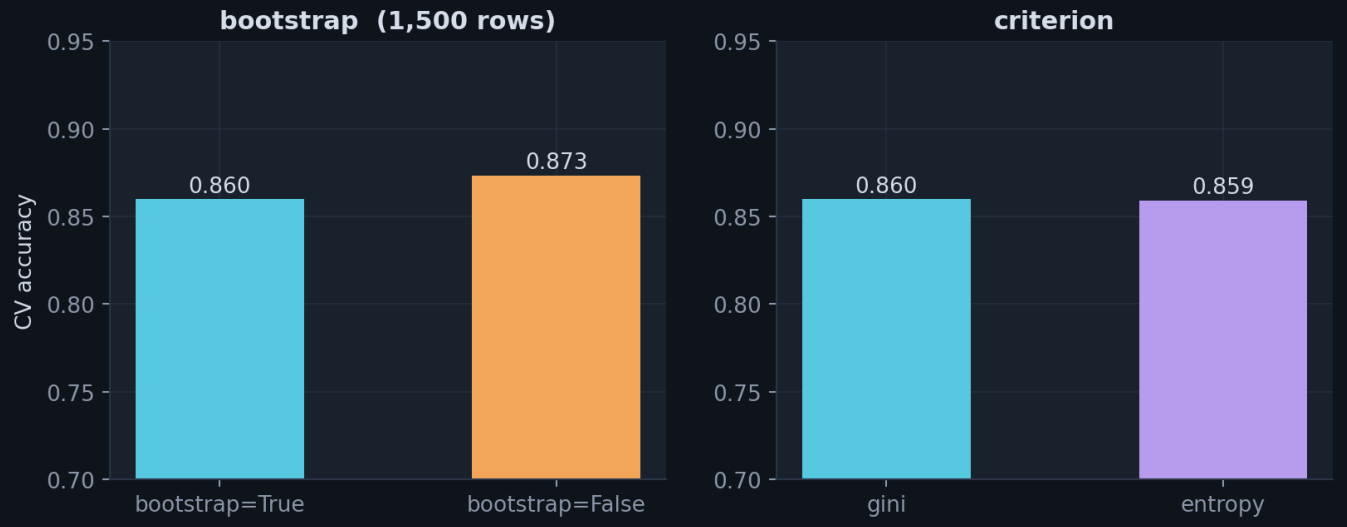


Figure 14 (measured) — Left: bootstrap True vs False. Right: Gini vs entropy (86.0% vs 85.9%) — practically identical, so `criterion` is usually the *least* important knob.

3.7 How the knobs interact

Hyperparameter	Increase it →	Effect on the model
`n_estimators`	more trees	Lower variance, more stable; slower. Rarely overfits.
`max_depth`	deeper trees	More complex (↑ variance, ↓ bias). Cap it to regularise.
`min_samples_split`	larger value	Simpler trees (↓ variance, ↑ bias).
`min_samples_leaf`	larger value	Simpler, smoother predictions; strongest regulariser.
`max_features`	more features per split	Stronger but more correlated trees; slower.
`bootstrap`	True → False	Trees see all rows: less diversity, no OOB score.

INTERACTIONS

Knobs that fight overfitting together: small `max_depth` + large `min_samples_leaf` + large `min_samples_split` do overlapping jobs. Using all three strongly may over-constrain the model. That is exactly why we search them *jointly* with GridSearchCV instead of one at a time.

Knobs that tune the crowd: `n_estimators`, `max_features` and `bootstrap` control variance reduction through the $\rho\sigma^2 + (1 - \rho)\sigma^2/B$ formula.

Part 4 — GridSearchCV: Tuning Automatically (Practice)

4.1 Why we need a search

You could try settings by hand: change `max_depth`, retrain, look at the score, repeat. This quickly becomes unmanageable because the best value of one knob depends on the others (Section 3.7). A **grid search** automates it: you list candidate values per hyperparameter, and the computer tries *every combination*, scores each fairly, and tells you the winner.

> **ANALOGY**
> Ordering a set meal: 3 starters × 4 mains × 2 desserts = 24 possible meals. Grid search tastes *all 24* and reports the best. Its weakness is also visible here — add a fourth course with 5 options and you have 120 meals to taste.

4.2 Cross-validation: how each combination is scored fairly

How do we score a combination honestly? If we test on the same data we trained on, an overfit model looks perfect. If we tune on the final test set, the test set stops being a fair exam (this is **data leakage**). The solution is **k-fold cross-validation** on the training set:

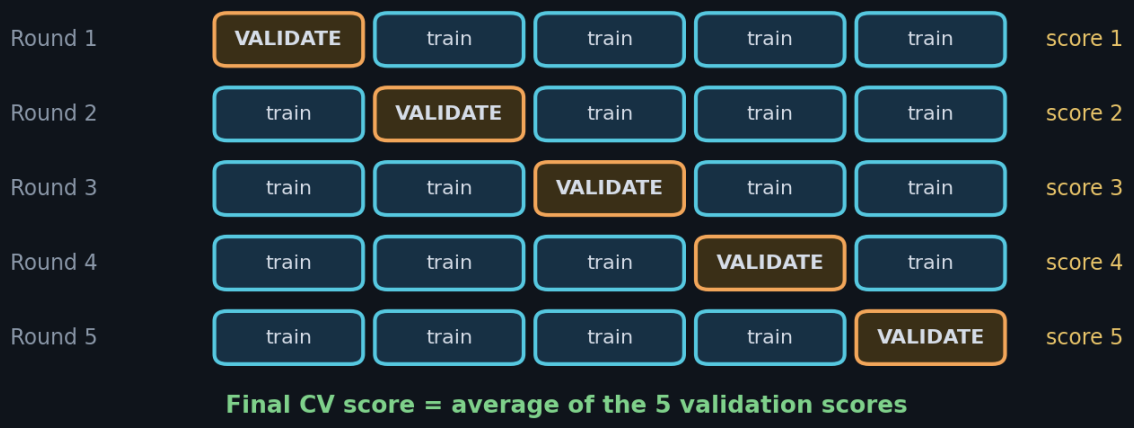


Figure 15 — 5-fold cross-validation: the training data is cut into 5 parts; each part takes one turn as the validation set while the other 4 train the model.

$$\text{CV score} = \frac{1}{k}\sum_{i=1}^{k} \text{score}_i$$

In your code `cv=5`, so k = 5 and each hyperparameter combination is trained 5 times.

Data part	Used for	Touched when?
Training folds	Fit the model inside each CV round	During search
Validation fold	Score one CV round	During search
Test set	Final, single, honest exam	**Only once, at the very end**

4.3 Anatomy of GridSearchCV

Argument	Meaning	Typical value
`estimator`	The base model to tune.	`RandomForestClassifier(random_state=42)`
`param_grid`	Dictionary: name → list of candidate values.	See code below
`cv`	Number of folds (or a splitter object).	`5`
`scoring`	Metric to maximise (Section 4.5).	`'accuracy'`, `'f1'`, `'neg_mean_squared_error'`
`n_jobs`	CPU cores to use; -1 = all of them.	`-1`
`verbose`	How much progress to print.	`1`
`refit`	Retrain the best combo on all training data (default True).	`True`

Attribute after `.fit()`	What it gives you
`best_params_`	The winning combination (a dictionary).
`best_score_`	Mean CV score of the winner. **It is a validation score, not the final test score.**
`best_estimator_`	The model refit with the winning parameters — use it for predictions.
`cv_results_`	Everything: every combination's mean/std score, rank and timing (convert to a DataFrame).

4.4 The grid-size maths

$$N_{fits} = k \times \prod_j n_j$$

$$\text{Total time} \approx \frac{N_{fits} \times \bar{t}_{fit}}{\text{cores}}$$

n_j = number of values for hyperparameter j, k = CV folds, $\bar{t}$ = average time for one fit.

Using the grid from your notes: 4 × 4 × 3 × 3 × 3 × 2 × 2 = **1,728 combinations**, times 5 folds = **8,640 model fits** — and every fit trains up to 500 trees.

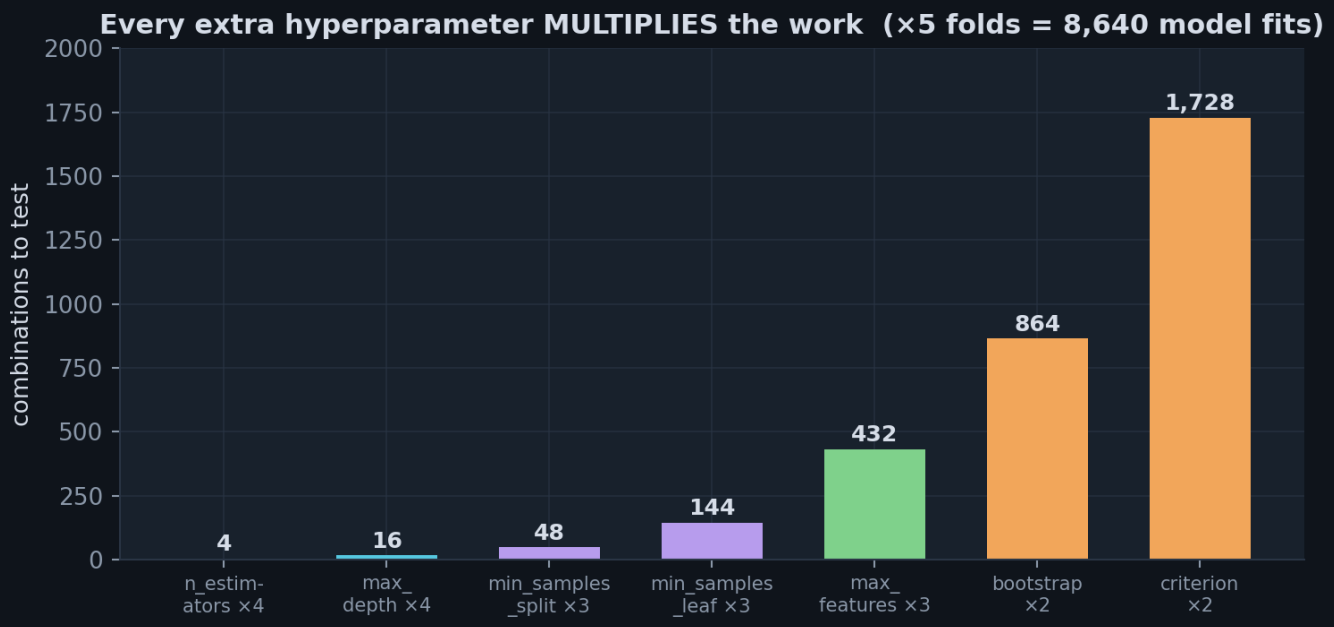


Figure 16 — Each additional hyperparameter multiplies the number of combinations. This is why large grids become slow quickly.

WATCH OUT

In our small demo (Section 4.7) one fit took about 0.45 s on a single core. At that speed, 8,640 fits would take roughly 65 minutes on one core — and much longer for the 300- and 500-tree settings. Fixes: use `n_jobs=-1`, shrink the grid (Part 6 shows coarse-to-fine search), or use RandomizedSearchCV.

4.5 Classification or regression? Choosing the scoring metric

The `scoring` argument tells GridSearchCV what "best" means, so it depends on your problem type. In your own words: "are you applying this to classification or regression?" — here are the choices for both.

Problem	Metric	`scoring=`	When to use
Classification	Accuracy	`'accuracy'`	Classes are balanced.
Classification	F1 score	`'f1'` (`'f1_macro'` for multi-class)	Imbalanced classes; balances precision and recall.
Classification	ROC-AUC	`'roc_auc'`	Ranking quality across all thresholds.
Classification	Balanced accuracy	`'balanced_accuracy'`	Imbalanced classes, easy to explain.
Regression	MSE (negated)	`'neg_mean_squared_error'`	Penalise large errors heavily.
Regression	RMSE (negated)	`'neg_root_mean_squared_error'`	Error in the same unit as the target.
Regression	MAE (negated)	`'neg_mean_absolute_error'`	Robust to outliers.
Regression	R^2	`'r2'`	Share of variance explained (1 = perfect).

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} \quad \text{Precision} = \frac{TP}{TP+FP} \quad \text{Recall} = \frac{TP}{TP+FN} \quad F_1 = \frac{2PR}{P+R}$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum(y_j - \hat{y}_j)^2} \qquad R^2 = 1 - \frac{\sum(y_j - \hat{y}_j)^2}{\sum(y_j - \bar{y})^2}$$

TP/TN = correct positives/negatives, FP/FN = false alarms/misses. P = precision, R = recall.

NOTE

Why “neg_”? GridSearchCV always **maximises** the score. Error metrics (smaller is better) are therefore negated: −MSE closest to 0 is the best, and `best_score_` will print as a negative number such as −12.4. That is normal. Multiply by −1 to read the actual MSE.

4.6 The complete GridSearchCV code for Random Forest

This is the full workflow with **every** hyperparameter from your notes. Read the numbered comments — each step is explained afterwards.

Python — GridSearchCV with Random Forest (classification)

```
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# 1. Load data (replace the next line with: data = pd.read_csv('your_dataset.csv'))
X, y = make_classification(n_samples=1500, n_features=20, n_informative=6,
                           n_redundant=4, flip_y=0.05, class_sep=0.8, random_state=42)

# 2. Split: the test set stays locked away until the very end
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Base model (random_state makes results reproducible)
rf = RandomForestClassifier(random_state=42)

# 4. The hyperparameter grid: every parameter from your notes
param_grid = {
    'n_estimators':      [100, 200, 300, 500],
    'max_depth':         [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf':  [1, 2, 4],
    'max_features':      ['sqrt', 'log2', None],
    'bootstrap':         [True, False],
    'criterion':         ['gini', 'entropy'],
}   # 4*4*3*3*3*2*2 = 1,728 combinations  x 5 folds = 8,640 fits

# 5. Configure and run the search
grid_search = GridSearchCV(estimator=rf, param_grid=param_grid, cv=5,
                           scoring='accuracy', n_jobs=-1, verbose=1)
grid_search.fit(X_train, y_train)

# 6. Best settings found by cross-validation
print("Best CV accuracy   :", round(grid_search.best_score_, 4))
print("Best hyperparameters:", grid_search.best_params_)

# 7. One honest final exam on the untouched test set
best_rf = grid_search.best_estimator_
y_pred = best_rf.predict(X_test)
print("Test accuracy:", round(accuracy_score(y_test, y_pred), 4))
print(classification_report(y_test, y_pred))
```

Regression version

For a numeric target, change three things: the model, the `criterion` values and the `scoring` metric:

Python — regression variant

```
from sklearn.ensemble import RandomForestRegressor

param_grid = {
    'n_estimators':      [100, 200, 500],
    'max_depth':         [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf':  [1, 2, 4],
    'max_features':      ['sqrt', 'log2', None],
    'bootstrap':         [True, False],
    'criterion':         ['squared_error', 'absolute_error'],   # absolute_error is SLOW
}
grid_search = GridSearchCV(RandomForestRegressor(random_state=42), param_grid, cv=5,
                           scoring='neg_mean_squared_error', n_jobs=-1, verbose=1)
```

4.7 Reading the results — a real run

The full grid is too big to run in a demo, so we ran a smaller one (36 combinations × 5 folds = 180 fits) with the same code structure on our dataset:

Python — the demo grid

```
param_grid = {'n_estimators': [100, 200], 'max_depth': [None, 10, 20],
              'min_samples_leaf': [1, 2, 4], 'max_features': ['sqrt', 'log2']}
grid_search = GridSearchCV(RandomForestClassifier(random_state=42), param_grid,
                           cv=5, scoring='accuracy', n_jobs=-1).fit(X_train, y_train)
```

Output (measured, this document)

```
Best CV accuracy   : 0.8683
Best hyperparameters: {'max_depth': 10, 'max_features': 'sqrt',
                       'min_samples_leaf': 2, 'n_estimators': 100}
Test accuracy (tuned model)  : 0.8267
Test accuracy (default model): 0.8433
```

To inspect *all* combinations, turn `cv_results_` into a table and sort by rank:

```
results = pd.DataFrame(grid_search.cv_results_)
cols = ['param_max_depth', 'param_min_samples_leaf', 'mean_test_score',
        'std_test_score', 'rank_test_score']
print(results[cols].sort_values('rank_test_score').head(5))
```

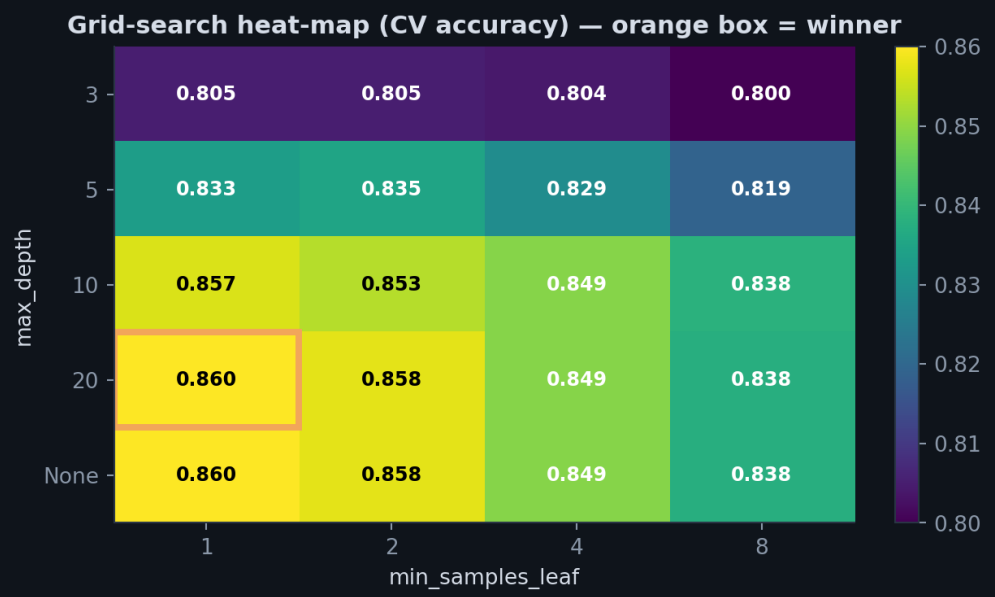


Figure 17 (measured) — A heat-map of `max_depth` × `min_samples_leaf` on our data (100 trees). The best cell reaches 86.0%; notice how many neighbouring cells are within a fraction of a point. A broad plateau is common.

WATCH OUT

A reality check. The tuned forest scored 86.8% in cross-validation, yet on the 300-row test set it scored 82.7% — slightly *below* the untuned default (84.3%). The difference is about 5 rows out of 300, which is within random noise. Lessons: (1) default forests are already strong, so expect modest gains; (2) never trust a tiny difference — look at `std_test_score` and prefer simpler settings when scores tie; (3) the untouched test set exists precisely to catch over-optimistic tuning.

Part 5 — Optimising Decision Trees: Faster, Smaller, More Accurate

Everything in a Random Forest is built from single decision trees, so making one tree better makes the forest better. There are two goals that sometimes pull in opposite directions: **predictive efficiency** (generalise well, higher accuracy on unseen data) and **computational efficiency** (train fast, use little memory). Your notes contain two lists of tricks. We start with the **easy version** (plain-language analogies) and then go through the **technical version** with maths, code and measurements.

5.1 The easy version

Simplifying the data

Pre-bin continuous variables (bucketing)
Instead of making the computer check every exact number (e.g., ages 21, 22, 23.5), group them into broader buckets (e.g., “Teens”, “20s”, “30s”). It is significantly faster for the algorithm to choose between a few categories than thousands of exact decimals.

```
df['age_group'] = pd.cut(df['age'], bins=[0, 19, 29, 39, 120], labels=['teen', '20s', '30s', '40+'])
```

Aggressive feature selection (dropping useless columns)
Don’t feed the model irrelevant data. If you are predicting house prices, square footage matters, but the homeowner’s favourite colour does not. Deleting useless columns stops the computer from wasting time evaluating them.

```
X = X.drop(columns=['favourite_colour', 'row_id'])
```

Downcast data types (memory compression)
This is like zipping a large file so it loads faster. Telling the computer to store numbers in smaller memory spaces (32-bit instead of 64-bit) halves the memory usage and prevents the program from crashing on large datasets.

```
X = X.astype('float32')
```

Setting boundaries on the tree

Enforce max_depth early (set a height limit)
If you do not set a limit, the tree will keep branching out until it has a specific rule for every single row in your data. Capping the depth at 5 or 10 forces the tree to stop early, saving time and making it learn general rules instead of memorising the data.

```
DecisionTreeClassifier(max_depth=8)
```

Increase min_samples_split (ignore tiny groups)
Tell the tree to stop branching if a group gets too small. If a branch only has 3 data points left in it, it is not worth the mathematical effort to split it again. Requiring at least 50 or 100 points to justify a split can save large amounts of time on big datasets.

```
DecisionTreeClassifier(min_samples_split=50)
```

Use max_features (limit the choices)
Instead of making the tree look at all 100 columns to make every single decision, tell it to randomly look at only 10 columns at each step. Doing less math at every branch drastically speeds up the training.

`DecisionTreeClassifier(max_features=10)` # or 'sqrt'

Advanced shortcuts

Cost-complexity pruning (trimming the fat)
Let the tree grow out, but then use an algorithm to act like scissors and snip off the tiny, useless branches at the bottom that do not actually improve your accuracy.

`DecisionTreeClassifier(ccp_alpha=0.002)`

Switch to histogram-based trees (upgrade your engine)
If you have over 10,000 rows of data, the standard decision tree tool is too slow. Switching to an upgraded algorithm (like LightGBM or scikit-learn's HistGradientBoosting) automatically buckets the data in the background, processing massive datasets in a fraction of the time.

`from sklearn.ensemble import HistGradientBoostingClassifier`

NOTE
`HistGradientBoostingClassifier` is not a single tree: it is a *boosted* ensemble of trees (each new tree corrects the errors of the previous ones) that uses histogram binning internally. It is the natural "next step" after Random Forest, which is why it is so fast and accurate on big tables. Compare it fairly against a forest, not against a single tree.

5.2 The technical version

A. Predictive efficiency (generalisation and accuracy)

Cost-complexity pruning (post-pruning)
Apply the `ccp_alpha` parameter to penalise complex tree structures. This algorithm grows a massive tree and then systematically trims the weakest terminal branches, retaining only the splits that contribute significantly to reducing overall impurity.

Mathematically, pruning minimises a penalised error: the tree's impurity R(T) plus α times the number of leaves |T|:

$$R_\alpha(T) = R(T) + \alpha\,|T| \qquad \alpha_{eff}(t) = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

A branch T_t is cut when α ≥ its effective α: the impurity it saves is not worth the extra leaves. α = 0 → no pruning; large α → a single node.

Python — exploring the pruning path

```
path = DecisionTreeClassifier(random_state=0).cost_complexity_pruning_path(X_train, y_train)
alphas = path.ccp_alphas[:-1]            # the last alpha would prune the tree to a single node
for a in alphas[::10]:
    tree = DecisionTreeClassifier(random_state=0, ccp_alpha=a).fit(X_train, y_train)
    print(round(a, 5), tree.get_n_leaves(), round(tree.score(X_test, y_test), 3))
```

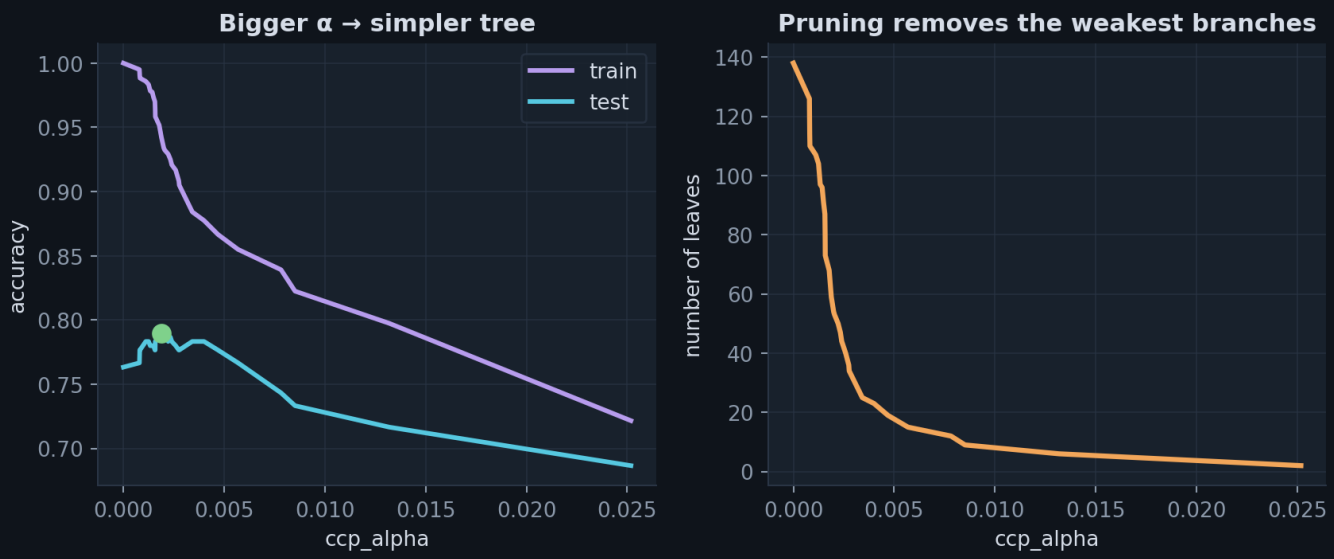


Figure 18 (measured) — As α grows the tree shrinks from 138 leaves; the best test accuracy (79.0% at α ≈ 0.0019, 59 leaves) beats the unpruned tree (76.3%). In practice choose α with cross-validation (it is just another GridSearchCV parameter).

Aggressive pre-pruning

Force the tree to stop growing before it memorises the training data. Set a strict `max_depth` (e.g., 5 to 12) combined with conservative `min_samples_leaf` thresholds to prevent the creation of single-data-point leaves that only capture noise.

```
DecisionTreeClassifier(max_depth=8, min_samples_leaf=5)
```

PRO TIP

Pre-pruning vs post-pruning: pre-pruning stops growth early (fast, but may stop just before a useful split); post-pruning grows fully and cuts back (slower, usually better-informed). Beginners start with pre-pruning; pros often tune both.

Class weight balancing

If your target variable is unevenly distributed, set `class_weight='balanced'`. This automatically adjusts weights inversely proportional to class frequencies, forcing the algorithm to heavily penalise errors on minority classes without requiring expensive synthetic data generation like SMOTE.

```
RandomForestClassifier(class_weight='balanced')
```

$$w_c = \frac{n_{samples}}{n_{classes} \times n_c}$$

Example: 1,000 rows, 900 “No” and 100 “Yes” → w_{No} = 1000/(2×900) ≈ 0.56, w_{Yes} = 1000/(2×100) = 5.0. A mistake on a “Yes” row now costs 9× more.

Targeted feature selection

Drop highly correlated or collinear features before training. While decision trees are somewhat robust to collinearity, redundant variables dilute feature importance scores and create unnecessary split evaluations.

Python — drop one column from every highly correlated pair

```python
corr = X_df.corr().abs()
upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))   # upper triangle only
to_drop = [col for col in upper.columns if (upper[col] > 0.9).any()] # keep one of each pair
X_df = X_df.drop(columns=to_drop)
```

B. Computational efficiency (speed and memory)

Continuous feature discretisation (binning)
Convert continuous numeric variables into discrete intervals (e.g., transforming a continuous “Age” column into 10 categorical bins). Finding the optimal split for continuous variables is the most computationally expensive operation in tree building. Binning drastically reduces the mathematical search space.

Why? For a feature with u distinct values a tree must test $u - 1$ possible thresholds at every node. With 10 bins there are only 9.

Python — 10 equal-population bins per feature

```
from sklearn.preprocessing import KBinsDiscretizer
binner = KBinsDiscretizer(n_bins=10, encode='ordinal', strategy='quantile')
X_binned = binner.fit_transform(X_train)      # fit on TRAIN only, then transform test
```

Dimensionality reduction
Remove low-variance or irrelevant columns entirely. At every internal node, the algorithm must evaluate potential splits across all provided features. Trimming the dataset width linearly decreases training time.

```
from sklearn.feature_selection import VarianceThreshold
X_train = VarianceThreshold(threshold=0.01).fit_transform(X_train)   # drop near-constant columns
```

Data type downcasting
Convert 64-bit floating-point numbers (float64) and integers (int64) in your dataframe to 32-bit or 16-bit formats. This directly halves the memory footprint required to hold the data structures during the recursive splitting process.

$$\text{Memory} = \text{rows} \times \text{columns} \times \text{bytes per value}$$

1,000,000 rows × 20 columns × 8 bytes (float64) = 160 MB; float32 → 80 MB; float16 → 40 MB.

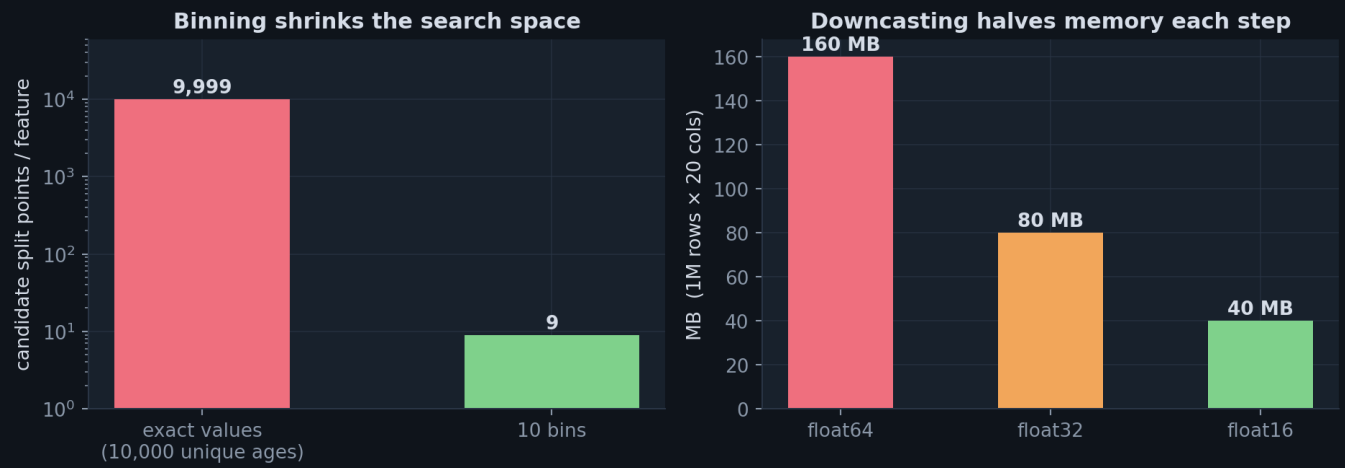


Figure 19 — Left: candidate thresholds a tree must test for one feature. Right: memory for a 1M × 20 table (float16 is lossy — use it only if the precision loss is acceptable).

Max features constraint
Set `max_features` to a fraction (e.g., 0.8) or `'sqrt'` rather than evaluating 100% of the features at every split. While typically used in ensembles, applying this to a standalone decision tree speeds up training and forces the algorithm to explore alternative structural paths.

C. Measuring the shortcuts

We timed these tricks on a larger dataset (40,000 rows × 40 features; 30,000 for training) so the effect is visible:

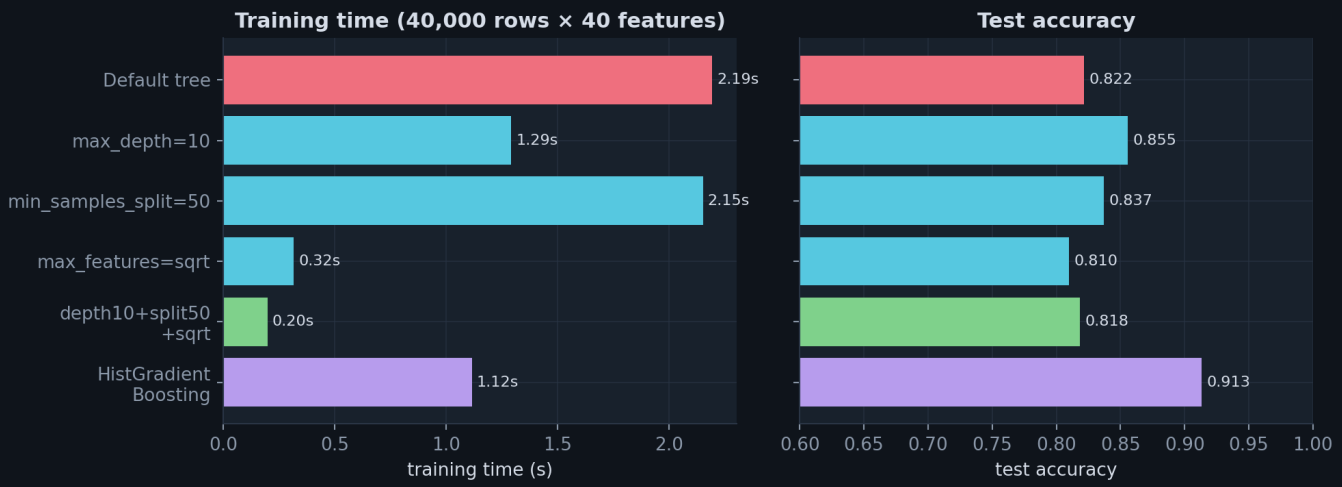


Figure 20 (measured) — Training time and test accuracy for several shortcuts on the same data.

Setting	Train time	Test accuracy
Default tree	2.19 s	82.2%
max_depth=10	1.29 s	85.5%
min_samples_split=50	2.15 s	83.7%
max_features=sqrt	0.32 s	81.0%
depth10+split50 +sqrt	0.20 s	81.8%
HistGradient Boosting	1.12 s	91.3%

WHAT THE EXPERIMENT SHOWS

Read the numbers honestly: capping `max_depth` at 10 cut the time from 2.19s to 1.29s *and* raised accuracy (82.2% → 85.5%) because it removed overfitting. Restricting `max_features` to 'sqrt' was the biggest speed-up (0.32s). `min_samples_split=50` alone saved almost nothing here (2.15s) — so not every trick pays off equally on every dataset; measure on yours. Combining depth + split + sqrt was the fastest (0.20s) with similar accuracy to the default tree.

The histogram-based booster took 1.12s and reached 91.3% — much higher accuracy, but remember it is an ensemble (boosting), so the comparison is not like-for-like with a single tree.

5.3 Your full Decision Tree GridSearchCV code

Here is the complete template from your notes, exactly as written, with its comments. It tunes a single `DecisionTreeClassifier` using five hyperparameters:

Python — your template, unchanged (1/2: steps 1–4)

```python
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# 1. Load Data (Using a synthetic dataset for this standalone template)
# Replace the next line with: data = pd.read_csv('your_dataset.csv')
# X = data.drop('target_column', axis=1)
# y = data['target_column']
X, y = make_classification(n_samples=1000, n_features=20, random_state=42)

# 2. Split into Training and Testing Sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Initialize Base Model
# Setting random_state ensures your results are reproducible every time you run the script
dt = DecisionTreeClassifier(random_state=42)

# 4. Define the Hyperparameter Grid
# These are the exact parameters discussed previously, mapped out for testing
param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [None, 5, 10, 15],
    'min_samples_split': [2, 10, 50],
    'min_samples_leaf': [1, 5, 10],
    'max_features': [None, 'sqrt']
}
```

Python — (2/2: steps 5–8)

```python
# 5. Configure GridSearchCV
# cv=5: Splits training data into 5 folds for cross-validation
# n_jobs=-1: Utilizes all available CPU cores to speed up the grid search
grid_search = GridSearchCV(
    estimator=dt,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1 # Prints progress updates to the console
)

# 6. Execute the Grid Search
print("Starting Grid Search...")
grid_search.fit(X_train, y_train)

# 7. Extract Best Parameters
print("\n--- Tuning Results ---")
print(f"Best Training Accuracy: {grid_search.best_score_:.4f}")
print(f"Best Hyperparameters: {grid_search.best_params_}")

# 8. Evaluate on Unseen Test Data
best_dt_model = grid_search.best_estimator_
y_pred = best_dt_model.predict(X_test)

print("\n--- Test Set Evaluation ---")
print(f"Test Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Step-by-step explanation

Step	What happens	Why it matters
1 Load data	Creates 1,000 synthetic rows with 20 features. Replace with your CSV.	Defines X (features) and y (target).
2 Split	80% training, 20% test (`test_size=0.2`).	The test set is the final exam — locked away.
3 Base model	An untuned `DecisionTreeClassifier`.	`random_state=42` makes the result repeatable.
4 Grid	5 hyperparameters: 2 × 4 × 3 × 3 × 2 = **144** combinations.	The "menu" GridSearchCV will try.
5 Configure	`cv=5` → 5 × 144 = **720 fits**; `n_jobs=-1` uses all cores.	Controls fairness (CV) and speed (cores).
6 Search	`fit()` trains and cross-validates every combination.	The expensive step.
7 Best params	Prints the winning combination and its CV score.	Your tuned settings.
8 Evaluate	Predicts on the untouched test set; prints accuracy and a report.	The honest performance number.

Real output from running exactly this code:

Output (measured, this document)

```
Starting Grid Search...
Fitting 5 folds for each of 144 candidates, totalling 720 fits

--- Tuning Results ---
Best Training Accuracy: 0.9050
Best Hyperparameters: {'criterion': 'entropy', 'max_depth': None,
                       'max_features': None, 'min_samples_leaf': 5,
                       'min_samples_split': 50}

--- Test Set Evaluation ---
Test Accuracy: 0.8800

Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.95      0.88        93
           1       0.95      0.82      0.88       107

    accuracy                           0.88       200
   macro avg       0.88      0.88      0.88       200
weighted avg       0.89      0.88      0.88       200
```

How to read the classification report. For each class, *precision* = of the rows predicted as this class, how many really were; *recall* = of the rows that really are this class, how many were found; *f1-score* = their harmonic mean; *support* = number of test rows of that class. "Macro avg" is the plain average of the classes; "weighted avg" weights by support.

FROM YOUR NOTES

To use your own data: Delete the `make_classification` line and uncomment the `pd.read_csv()` section at the top, pointing it to your specific CSV file and target column name.

To change the goal: If you are predicting a continuous number (like house prices) instead of categories, change `DecisionTreeClassifier` to `DecisionTreeRegressor`, and change the `scoring` parameter in GridSearchCV from `'accuracy'` to `'neg_mean_squared_error'`. (Also use regression criteria such as `'squared_error'` instead of `'gini'` / `'entropy'` in the grid, and remove `classification_report` — it is for classes only.)

WATCH OUT

A small label fix. The template prints “Best Training Accuracy”, but `best_score_` is actually the **mean cross-validation (validation) accuracy** of the best combination, not the training accuracy. Training accuracy of a tree like this is usually 100%. Consider renaming the label to “Best CV accuracy” so you never confuse the two. Also notice the winning tree used `max_depth=None` — depth was controlled instead by `min_samples_split=50` and `min_samples_leaf=5`; pruning knobs can substitute for each other.

Part 6 — Professional Workflow (Pro)

6.1 Beyond the full grid: smarter searches

Method	Idea	Use when	Cost
`GridSearchCV`	Try every combination.	Small grids (≲ a few hundred combos); final fine-tuning.	Grows multiplicatively.
`RandomizedSearchCV`	Sample a fixed number (`n_iter`) of random combinations from lists or distributions.	Many hyperparameters or wide ranges; limited time.	You choose it: `n_iter × cv` fits.
`HalvingGridSearchCV`	Start all candidates on a little data, keep the best third, give them more data, repeat.	Large data and large grids.	Much lower than a full grid.

Python — RandomizedSearchCV

```
from scipy.stats import randint
from sklearn.model_selection import RandomizedSearchCV

param_dist = {
    'n_estimators':      randint(100, 600),          # any integer from 100 to 599
    'max_depth':         [None, 10, 20, 30],
    'min_samples_split': randint(2, 20),
    'min_samples_leaf':  randint(1, 10),
    'max_features':      ['sqrt', 'log2', 0.5, None],
    'bootstrap':         [True, False],
}
rs = RandomizedSearchCV(RandomForestClassifier(random_state=42), param_dist, n_iter=60,
                        cv=5, scoring='accuracy', n_jobs=-1, random_state=42, verbose=1)
rs.fit(X_train, y_train)          # only 60 x 5 = 300 fits instead of thousands
print(rs.best_params_, rs.best_score_)
```

Python — successive halving

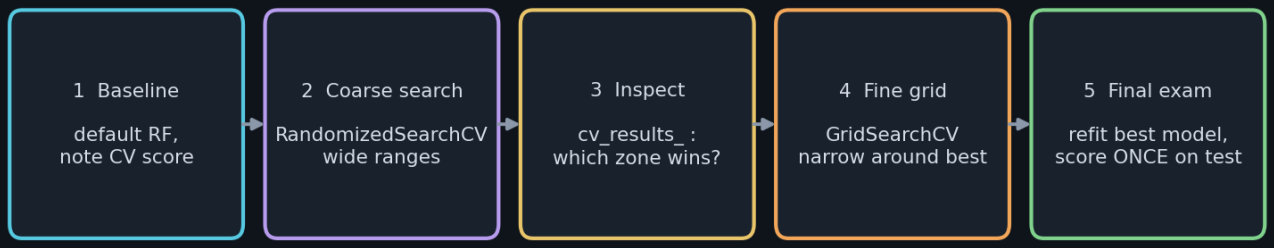
```
from sklearn.experimental import enable_halving_search_cv   # noqa: required import
from sklearn.model_selection import HalvingGridSearchCV

hs = HalvingGridSearchCV(RandomForestClassifier(random_state=42), param_grid, cv=5,
                         factor=3, n_jobs=-1, random_state=42)
hs.fit(X_train, y_train)
```

KEY IDEA

Random search works well because usually only a few hyperparameters strongly matter. A grid wastes many trials repeating the same few values of the important ones, whereas random sampling tries a new value of *every* parameter on each trial.

6.2 The coarse-to-fine workflow



Never touch the test set while tuning — it is only for step 5.

Figure 21 — A professional tuning loop that keeps the test set honest.

- **Baseline:** train a default forest, note the CV score. Everything must beat this.
- **Coarse search:** `RandomizedSearchCV` with wide ranges and 50–100 iterations, using 100–200 trees.
- **Inspect `cv_results_`:** which regions score best? Which parameters hardly matter (drop them from the next grid)?
- **Fine grid:** `GridSearchCV` over narrow ranges around the best region (2–3 values per parameter).
- **Final:** raise `n_estimators` (300–1000), refit on all training data, evaluate **once** on the test set.

6.3 Pipelines, data leakage and nested cross-validation

Leakage means information from the validation/test data sneaks into training. A classic mistake: fitting an imputer, scaler or feature selector on the *whole* dataset before cross-validation. The fix is a `Pipeline`, which re-fits every preprocessing step inside each fold. In the grid, parameters of a step are written as `stepname__parameter`:

Python — preprocessing inside the CV loop

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

pipe = Pipeline([
    ('impute', SimpleImputer(strategy='median')),
    ('model',  RandomForestClassifier(random_state=42)),
])
param_grid = {'model__n_estimators': [100, 200],
              'model__max_depth': [None, 10, 20],
              'model__min_samples_leaf': [1, 2, 4]}      # note the  step__param  naming
grid = GridSearchCV(pipe, param_grid, cv=5, scoring='f1', n_jobs=-1).fit(X_train, y_train)
```

Nested cross-validation. `best_score_` is slightly optimistic because the same folds both chose and scored the winner. To get an unbiased estimate of how a *tuned* model will perform, wrap the whole search inside an outer CV:

Python — nested CV

```
from sklearn.model_selection import cross_val_score, StratifiedKFold

inner = StratifiedKFold(n_splits=3, shuffle=True, random_state=1)
outer = StratifiedKFold(n_splits=5, shuffle=True, random_state=2)
search = GridSearchCV(rf, param_grid, cv=inner, scoring='accuracy', n_jobs=-1)
scores = cross_val_score(search, X, y, cv=outer)    # the search is re-run inside every outer fold
print(scores.mean(), scores.std())
```

6.4 Out-of-bag score: free validation

Recall the ≈ 36.8% of rows each tree never saw. Averaging, for every row, the votes of only those trees gives an **OOB score**, which approximates a cross-validation score at zero extra cost: one fit instead of five.

```
rf = RandomForestClassifier(n_estimators=300, oob_score=True, bootstrap=True, random_state=42)
rf.fit(X_train, y_train)
print(rf.oob_score_)
```

In our run the OOB accuracy was 86.7% versus 83.7% on the test set. They need not match exactly (different rows), but OOB is a quick, fair sanity check — especially to tune on huge data where a full 5-fold search is too expensive.

6.5 What did the forest learn? Feature importance

Impurity-based importance (built in) adds up, for each feature, how much impurity its splits removed, weighted by the number of samples reaching each split, then averages over trees:

$$\text{Imp}(f) = \frac{1}{B}\sum_{b=1}^{B} \sum_{t \in T_b:\ \text{split on } f} \frac{N_t}{N}\,\Delta I(t)$$

Fast, but biased toward features with many distinct values, and computed on training data.

Permutation importance is more trustworthy: shuffle one column of the test set and see how much the score drops. A big drop means the model relied on that feature.

```
from sklearn.inspection import permutation_importance
rf = RandomForestClassifier(n_estimators=300, random_state=0).fit(X_train, y_train)
print(rf.feature_importances_)                                  # impurity-based
perm = permutation_importance(rf, X_test, y_test, n_repeats=10, random_state=0)
print(perm.importances_mean)                                    # accuracy lost when shuffled
```

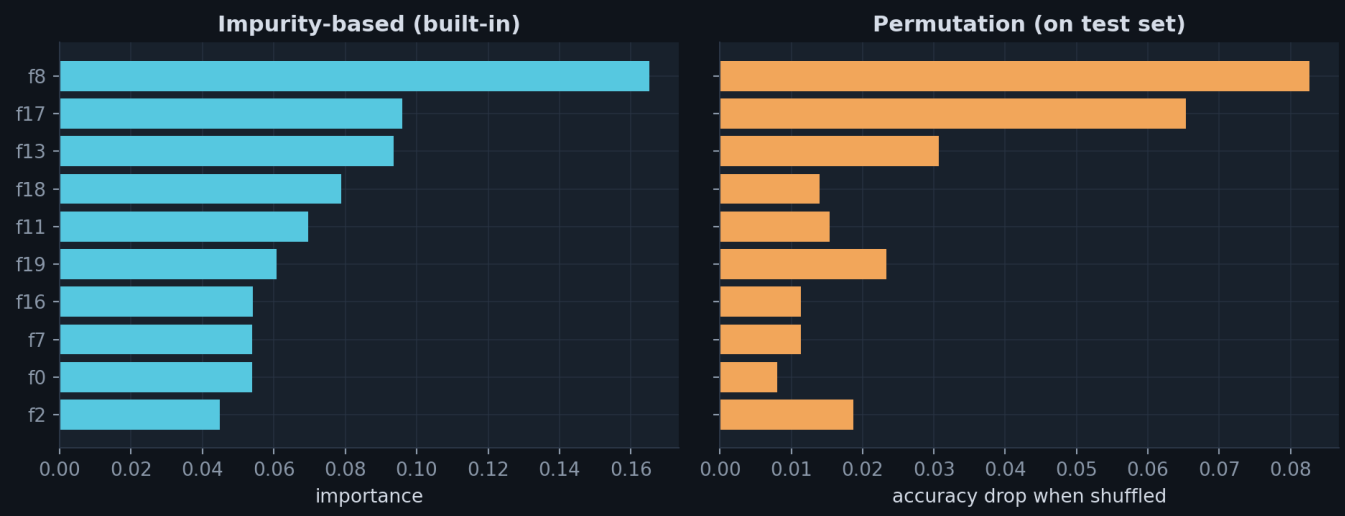


Figure 22 (measured) — Both methods agree on the top features, but permutation importance (right) shows the real drop in accuracy and ignores features that did not help on unseen data.

WATCH OUT

Importance shows what the model *uses*, not what *causes* the outcome. Correlated features split the credit between them, so one of a correlated pair can look unimportant.

6.6 Speed and memory playbook

Symptom	Try first	Then
Grid search takes hours	`n_jobs=-1`; smaller grid; `RandomizedSearchCV`; tune with 100 trees	Halving search; subsample rows for the search only
Training slow on big data	Lower `max_features`; cap `max_depth`; fewer trees	Bin features; downcast to float32; HistGradientBoosting
Out of memory	Downcast dtypes; drop useless columns	Cap depth / raise `min_samples_leaf` (smaller trees); fewer trees
Overfitting (train ≫ CV)	Raise `min_samples_leaf`; cap `max_depth`	Lower `max_features`; prune (`ccp_alpha`); more data
Underfitting (both low)	Remove depth caps; lower `min_samples_leaf`	Raise `max_features`; add features; try boosting
Minority class ignored	`class_weight='balanced'` (or `'balanced_subsample'`)	Score with F1 / ROC-AUC; stratified CV

6.7 Common mistakes

Mistake	Why it hurts	Fix
Tuning on the test set	The test score becomes optimistic — leakage.	Tune with CV on training data only; test once.
Scaling / imputing before the split	Information leaks into validation folds.	Use a `Pipeline`.
Reading `best_score_` as test accuracy	It is a validation score and slightly optimistic.	Report the test score (or nested CV).
Accuracy on imbalanced data	A model predicting only the majority class looks “great”.	Use F1 / ROC-AUC, `class_weight`, stratified folds.
One giant grid at once	Exponential cost.	Coarse random search → narrow grid.
Ignoring `std_test_score`	A 0.1-point gain may be pure noise.	Prefer the simpler model when scores tie.
Tuning `n_estimators` like the others	More trees almost never hurt; the search is wasted.	Fix it at 100–200 while tuning; raise at the end.
Forgetting `random_state`	Results change run to run.	Set it everywhere.

6.8 Master cheat-sheet

Hyperparameter	Default	Start grid	Too high / too low	Priority
`n_estimators`	100	[100, 200, 300, 500]	Low: noisy. High: slow only.	Low (set high)
`max_depth`	None	[None, 10, 20, 30]	High: memorise. Low: underfit.	High (esp. single tree)
`min_samples_split`	2	[2, 5, 10]	High: underfit.	Medium
`min_samples_leaf`	1	[1, 2, 4]	High: underfit. 1: noisy leaves.	High
`max_features`	'sqrt'	['sqrt', 'log2', None]	High: correlated, slow. Low: weak trees.	High
`bootstrap`	True	[True, False]	False: no OOB, less diversity.	Low
`criterion`	'gini'	['gini', 'entropy']	Rarely matters.	Lowest
`ccp_alpha`	0.0	[0, 0.001, 0.005, 0.01]	High: over-pruned.	Medium (single tree)
`class_weight`	None	[None, 'balanced']	—	When imbalanced

PRO TIP
Priority is a rule of thumb from experience and from the experiments in this document — on your data the order can change. The reliable strategy is the workflow of Section 6.2.

Situation	Reasonable starting point (not a rule)
Small data (< 2,000 rows)	200–500 trees; `min_samples_leaf` [1, 2, 4]; `max_features` ['sqrt', 'log2', 0.5]; test `bootstrap` False; use 5-fold or repeated CV.
Large data (> 100,000 rows)	Search with 100 trees and a row subsample; `RandomizedSearchCV`; cap `max_depth` (15–30); larger `min_samples_leaf` (5–20); consider HistGradientBoosting.
Noisy data	Higher `min_samples_leaf` (4–16), depth cap (5–12), lower `max_features`.
Imbalanced classes	`class_weight` ['balanced', 'balanced_subsample']; scoring `f1` / `roc_auc`; stratified CV.

6.9 Glossary

Term	Meaning
Ensemble	A model made by combining many models.
Bagging	Training models on bootstrap samples and aggregating their outputs.
Bootstrap sample	A random sample of N rows drawn with replacement.
Out-of-bag (OOB)	Rows a particular tree did not see during training (≈ 36.8%).
Impurity	How mixed the classes in a node are (Gini, entropy).
Information gain	Drop in impurity produced by a split.
Overfitting	Excellent on training data, poor on new data.
Underfitting	Too simple to capture the pattern; poor everywhere.
Bias	Error from overly simple assumptions.
Variance	Error from sensitivity to the specific training sample.
Hyperparameter	A setting chosen before training.
Cross-validation	Repeated train/validate rounds over k folds to estimate performance.
Grid search	Trying every combination of listed hyperparameter values.
Data leakage	Validation/test information influencing training.
Pruning	Removing weak branches to simplify a tree.
Regularisation	Any constraint that stops a model from becoming too complex.

6.10 Self-check quiz

Question	Answer
Why do the trees in a forest need to be *different*?	If they were identical, averaging would not reduce variance (ρ = 1 in the variance formula).
About what fraction of rows is in a bootstrap sample, and what is the rest called?	≈ 63.2% are in-bag; the other ≈ 36.8% are out-of-bag (OOB) rows.
Does increasing `n_estimators` cause overfitting?	Not typically; it stabilises the model but costs time and memory (linear).
Which is stricter: `min_samples_split` or `min_samples_leaf`?	`min_samples_leaf`, because it can veto a split even if the node is large enough to be considered.
What does `max_features=None` do to a Random Forest?	Removes feature randomness, making it plain bagged trees: more correlated and slower.
A grid has 3 × 4 × 5 values and `cv=5`. How many fits?	3 × 4 × 5 = 60 combinations × 5 folds = 300 fits (plus 1 refit).
Why is `best_score_` negative with `neg_mean_squared_error`?	Scores are maximised, so errors are negated; −MSE closest to 0 is best.
When should you look at the test set?	Once, at the very end, after all tuning is finished.

6.11 Summary and what to do next

- A Random Forest is many **different** trees (bootstrap rows + random features) whose votes are averaged; diversity is what makes it work.
- Seven knobs matter: crowd size (`n_estimators`), tree limits (`max_depth`, `min_samples_split`, `min_samples_leaf`), diversity (`max_features`, `bootstrap`) and the scoring rule (`criterion`).
- **GridSearchCV** tries every combination using cross-validation; the grid size multiplies fast, so search coarse → fine and keep the test set untouched.
- Single trees can be sped up and regularised with pruning, binning, downcasting, `class_weight` and `max_features`; histogram boosting is the scalable upgrade.
- Always measure: defaults are strong, small gains can be noise, and the test set is the judge.

NEXT STEPS

Practice plan: (1) Run the Section 4.6 code on your own CSV with the small demo grid. (2) Plot a validation curve for `max_depth` and `min_samples_leaf`. (3) Compare against `RandomizedSearchCV` with `n_iter=40`. (4) Try `HistGradientBoostingClassifier` as the next model to learn.